HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF ELECTRICAL AND ELECTRONICS ENGINEERING

LSI Design Contest 2026

Hardware–Software Co-Design of a Quantized HiFiC GAN Accelerator for Learned Image Compression on FPGA

EDABK_ADAM

Supervisor:
Prof. Nguyen Duc Minh
Dr. Hoang Phuong Chi

Students:
Truong Quoc Anh (20224430)
Do Viet Dung (20224340)
Nguyen Duc Phuc (20234032)
Nguyen Dac Hai Dang (202414009)

Hanoi, January 31, 2026

INFORMATION PAGE

1. **Team name:** EDABK_ADAM
2. **Task level of design:** Level 4 – Unlimited(Advanced GAN is also acceptable)
3. **Members' information:**

- **Truong Quoc Anh**
  - University: Hanoi University of Science and Technology
  - Cohort: 67
  - Address: No. 1 Dai Co Viet street, Hai Ba Trung, Hanoi, Vietnam
  - Mail address: anh.tq224430@sis.hust.edu.vn
  - Size of T-shirt: XXL
- **Do Viet Dung**
  - University: Hanoi University of Science and Technology
  - Cohort: 67
  - Address: No. 1 Dai Co Viet street, Hai Ba Trung, Hanoi, Vietnam
  - Mail address: dung.dv224340@sis.hust.edu.vn
  - Size of T-shirt: XL
- **Nguyen Duc Phuc**
  - University: Hanoi University of Science and Technology
  - Cohort: 68
  - Address: No. 1 Dai Co Viet street, Hai Ba Trung, Hanoi, Vietnam
  - Mail address: phuc.nd234032@sis.hust.edu.vn
  - Size of T-shirt: XL
- **Nguyen Dac Hai Dang**
  - University: Hanoi University of Science and Technology
  - Cohort: 69
  - Address: No. 1 Dai Co Viet street, Hai Ba Trung, Hanoi, Vietnam
  - Mail address: dang.ndh2414009@sis.hust.edu.vn
  - Size of T-shirt: L

INTRODUCTION PAGE

In today's digital era, the rapid growth of visual data and multimedia content has created increasing demands for efficient image compression techniques. Traditional image compression standards, while widely adopted, often struggle to achieve high perceptual quality at low bitrates. In response to these limitations, learned image compression based on deep learning has emerged as a promising solution, enabling data-driven optimization of compression and reconstruction processes.

Among learned compression approaches, Generative Adversarial Network (GAN)-based models have demonstrated superior performance in preserving visual quality, especially in low-bitrate scenarios. High-Fidelity Image Compression (HiFiC) is a representative GAN-based framework that leverages adversarial learning to enhance perceptual fidelity. However, the high computational complexity of such models presents significant challenges for practical deployment, particularly in real-time and embedded systems.

This project investigates a hardware–software co-design approach for accelerating learned image compression using a quantized HiFiC GAN model. By combining FPGA-based hardware acceleration with software execution, the proposed system aims to achieve an efficient balance between performance, resource utilization, and reconstruction quality, providing a practical foundation for deploying advanced image compression models in real-world applications.

I would like to express my gratitude to Prof. Nguyen Duc Minh and Dr. Hoang Phuong Chi for their enthusiastic assistance and support throughout the process of developing the idea and implementing this project. I also sincerely thank my colleagues in the EDABK laboratory at Hanoi University of Science and Technology, for their help during the project implementation.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

Recent advances in deep learning have significantly improved image compression by enabling learned models to outperform traditional hand-crafted standards in both rate–distortion efficiency and perceptual quality. In particular, Generative Adversarial Network (GAN)-based approaches such as High-Fidelity Image Compression (HiFiC) have demonstrated strong capability in preserving fine visual details at low bitrates. However, the high computational complexity of GAN-based models poses major challenges for real-time deployment and embedded systems.

This project presents a hardware–software co-design approach for learned image compression based on a quantized HiFiC GAN model. The proposed system targets the inference stage of the HiFiC pipeline, where computationally intensive components are accelerated on FPGA, while the remaining functions are executed in software. To enable efficient hardware implementation, the neural network is quantized to FP16 precision and implemented using High-Level Synthesis (HLS), allowing effective exploitation of parallelism and streamlined design development.

The accelerator architecture adopts a streaming dataflow model with parallel processing elements to efficiently map convolutional operations onto hardware. The design is implemented and evaluated on the ZCU104 FPGA platform, demonstrating feasible operating frequency, acceptable resource utilization, and numerical accuracy comparable to software reference results.

Experimental results indicate that the proposed hardware–software co-design framework provides an effective and practical solution for accelerating learned image compression models. This work highlights the potential of FPGA-based acceleration for advanced GAN-based image compression and serves as a foundation for future extensions toward more comprehensive hardware deployment.

CHAPTER 1. INTRODUCTION

In recent years, the rapid growth of multimedia content and high-resolution visual data has posed significant challenges in terms of storage, transmission, and real-time processing. Traditional image compression standards such as JPEG and JPEG2000, which rely on handcrafted transforms and fixed processing pipelines, have shown limitations when dealing with complex image statistics and high compression ratios. As a result, learned image compression based on deep learning techniques has emerged as a promising alternative, offering superior rate–distortion performance and improved adaptability to diverse image content.

Among these approaches, Generative Adversarial Network (GAN)-based models have demonstrated remarkable effectiveness in learned image compression, particularly in perceptual quality optimization. The High-Fidelity Image Compression (HiFiC) framework represents a state-of-the-art GAN-based solution, leveraging adversarial training to generate visually pleasing reconstructions at low bitrates [1]. By incorporating a generator–discriminator architecture, HiFiC is capable of preserving fine-grained textures and semantic details that are often lost in conventional compression schemes.

Despite their impressive compression performance, GAN-based models such as HiFiC introduce substantial computational complexity. The inference pipeline consists of deep convolutional networks with multiple residual blocks, high-dimensional feature maps, and intensive arithmetic operations. These characteristics make direct deployment on general-purpose processors inefficient for real-time or embedded applications, thereby motivating the use of hardware acceleration.

Field-Programmable Gate Arrays (FPGAs) provide an attractive platform for accelerating deep learning workloads due to their inherent parallelism, reconfigurability, and favorable performance–power trade-offs. Modern FPGA platforms further enable tight integration between programmable logic and embedded processors, facilitating a hardware–software co-design paradigm. In such systems, computationally intensive kernels can be offloaded to dedicated hardware accelerators, while control and less demanding tasks are executed in software, resulting in a balanced and scalable system architecture.

In this work, we present a hardware–software co-design approach for learned image compression based on a quantized HiFiC GAN model. The proposed design targets the inference stage of the HiFiC pipeline and accelerates its most computationally demanding components on FPGA using High-Level Synthesis (HLS). To achieve efficient hardware implementation, the neural network is quantized to FP16 precision, signifi-

cantly reducing hardware resource usage while maintaining acceptable numerical accuracy. The accelerator is integrated into a heterogeneous system where hardware and software components cooperate seamlessly to perform end-to-end image compression.

The main contributions of this project can be summarized as follows:

- A hardware–software co-design framework for HiFiC GAN inference targeting learned image compression applications.
- An FPGA-based accelerator architecture optimized for quantized GAN workloads using streaming dataflow and parallel processing elements.
- A practical implementation and evaluation on the ZCU104 FPGA platform, demonstrating feasible performance, resource efficiency, and numerical accuracy.

The remainder of this report is organized as follows:

- **Proposed Algorithm**: This chapter describes the HiFiC GAN inference pipeline, the quantization strategy, and the functional architecture of the proposed accelerator.
- **FPGA Implementation**: This chapter presents the hardware–software co-design methodology and details the FPGA implementation using HLS.
- **Results**: This chapter reports experimental results, including resource utilization, latency, throughput, and accuracy evaluation.
- **Conclusions and Future Development**: This chapter summarizes the main achievements of the project and discusses potential directions for future work.

At the end of this report, the consulted literature is presented in the References section, followed by appendixes that include additional implementation details, supporting materials, and definitions of acronyms and terminology used throughout the document.

CHAPTER 2. PROPOSED ALGORITHM

The development of the project is divided into different steps in order to easily focus each part of the whole system and make it work itself as a smaller system. This way, at the end of the project, the whole system will be merged using the acquired know-how from the previous steps. This working methodology has been selected because of the complexity of the work field and the lack of knowledge about it from the company.

Accordingly, this section is composed by first a description of the final system and its architecture and then all the steps required to complete the work.

2.1 Specifications and Flow Design

Specifications:

- Input data: Image with resolution 256×256 (RGB).
- Compression model: Quantized HiFiC
- Target platform: ZCU104 FPGA board.
- Design tools: Vitis_HLS for hardware synthesis and Vivado for system integration.

Flow Design:

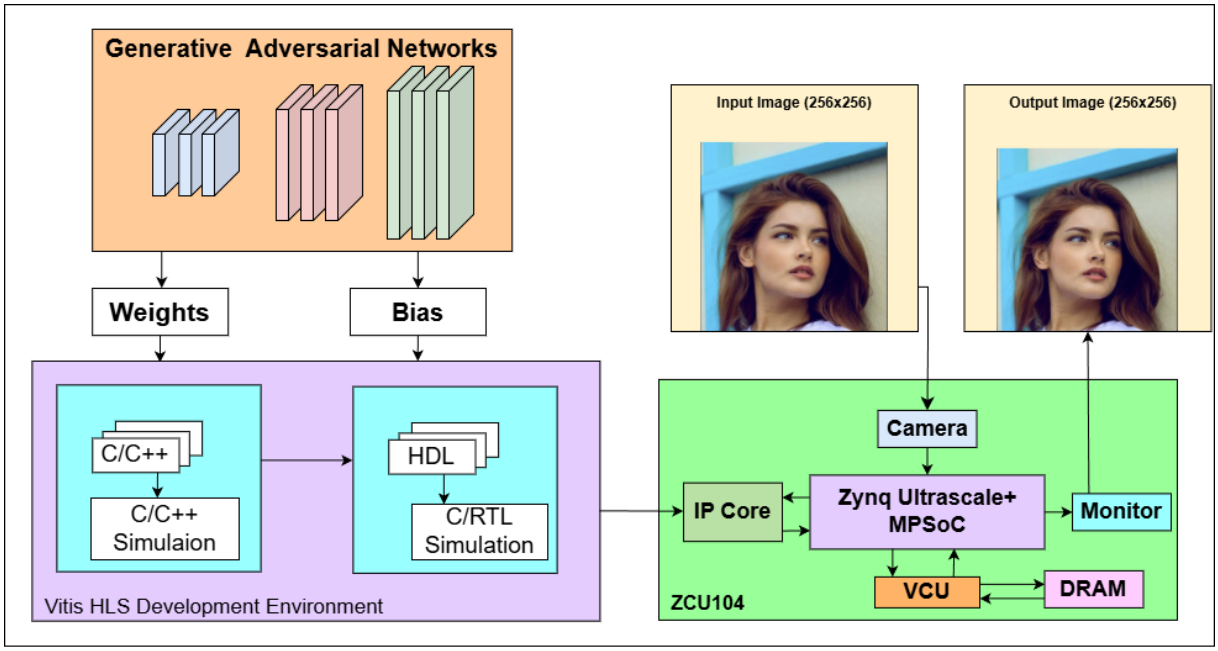

Figure 2.1 Overall flow design of the proposed image compression system

The proposed system follows a structured design flow that bridges algorithm development and hardware implementation, as illustrated in Figure 2.1. The overall workflow is divided into three main stages: model development, hardware synthesis, and system integration.

In the first stage, a convolutional neural network (CNN) model is designed and trained to perform image compression and reconstruction. This stage focuses on selecting an appropriate network architecture, tuning hyperparameters, and training the model using representative image datasets to achieve a balance between compression efficiency and reconstruction quality.

After successful training, the learned parameters of the CNN model, including weights and biases, are extracted and converted into a C++ representation. This step enables the transition from a high-level deep learning framework to a hardware-oriented implementation. The C++ model serves as a functional reference and forms the basis for hardware synthesis.

In the second stage, the C++ implementation is synthesized into hardware using the Vitis_HLS tool. Through high-level synthesis, the algorithmic description is transformed into a register-transfer level (RTL) design, resulting in a custom Intellectual Property (IP) core capable of performing CNN-based image compression. Hardware-oriented optimizations are applied at this stage to improve performance and resource efficiency.

In the final stage, the generated IP core is integrated into the ZCU104 system using the Vivado design suite. The hardware accelerator is connected with the processing system and external memory to form a complete image compression pipeline. This integration enables efficient execution of the CNN-based compression algorithm within an embedded system environment, demonstrating the feasibility of deploying deep learning-based image compression on FPGA platforms.

2.2 HiFiC GAN Architecture for Learned Image Compression

To perform learned image compression and reconstruction, this project adopts the High-Fidelity Image Compression (HiFiC) framework, which is based on a Generative Adversarial Network (GAN) architecture. As illustrated in Figure 2.2, the overall system consists of three main components: an encoder network, a hyperprior network, and a generator network. This architecture is designed to efficiently compress input images into compact latent representations while preserving high perceptual quality in the reconstructed output.

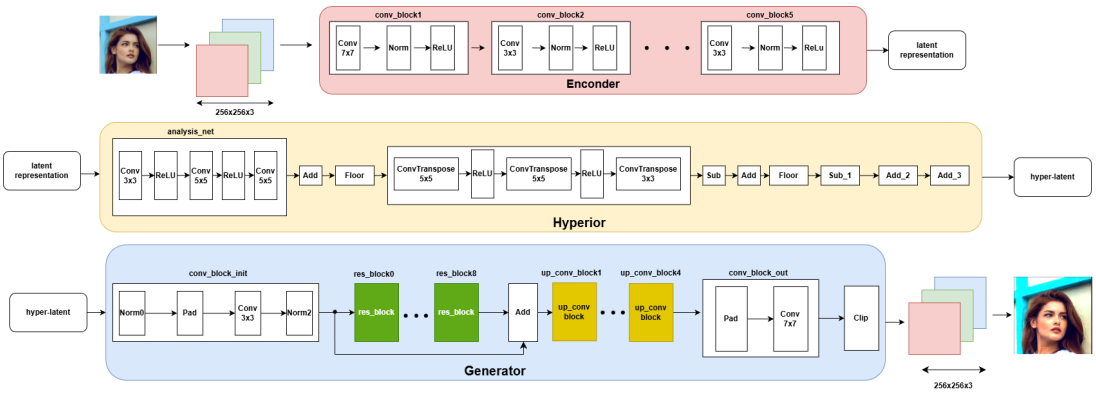

Figure 2.2 Overview of the HiFiC GAN-based learned image compression architecture

The encoder network processes the input image through a sequence of convolutional blocks composed of convolution, normalization, and activation layers. These blocks progressively extract hierarchical features and reduce spatial redundancy, producing a latent representation that captures the essential information of the input image. Unlike traditional autoencoders, the HiFiC encoder is optimized for perceptual quality rather than purely minimizing pixel-wise reconstruction error.

To further improve compression efficiency, a hyperprior model is employed to learn the statistical distribution of the latent representation. The hyperprior network generates side information, referred to as hyper-latent variables, which helps model the entropy of the latent space more accurately. This information is later used to facilitate efficient compression and reconstruction, forming a key component of the HiFiC framework.

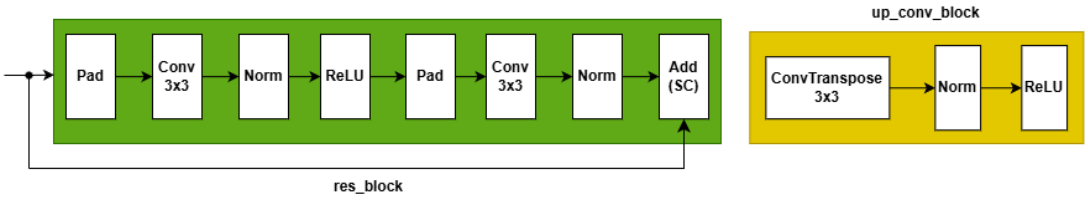

Figure 2.3 Detailed structure of the residual block and up-convolution block used in the generator network

The generator network reconstructs the output image from the latent and hyper-latent representations. As shown in Figure 2.3, the generator is composed of an initial convolutional block, followed by multiple residual blocks and up-convolution blocks. Each residual block contains convolutional layers with normalization and activation functions, along with a skip connection that enables stable training and efficient feature reuse. These residual blocks play a crucial role in refining image details and enhancing

perceptual quality.

The up-convolution blocks are implemented using transposed convolution layers combined with normalization and activation functions. These blocks progressively increase the spatial resolution of feature maps, enabling the reconstruction of high-resolution images from compact representations. A final convolutional block produces the reconstructed image, followed by a clipping operation to ensure valid pixel intensity ranges.

Among the components of the HiFiC architecture, the generator network contains the most computationally intensive operations, including multiple convolutional, residual, and upsampling layers. Therefore, in this work, the generator network is identified as the primary target for hardware acceleration in the proposed hardware–software co-design framework. The remaining components are handled in software to maintain flexibility and system-level integration.

Overall, the HiFiC GAN architecture provides an effective learned image compression framework that balances compression efficiency and perceptual quality. Its modular design enables selective hardware acceleration, making it well suited for deployment on FPGA-based embedded systems.

2.3 Mixed-Precision Quantization of HiFiC Model

The deployment of learned image compression models such as HiFiC on FPGA platforms introduces significant challenges related to memory bandwidth, storage capacity, and computational efficiency. To address these constraints, a mixed-precision quantization strategy is adopted, aiming to reduce numerical precision while preserving compression performance and perceptual quality.

Figure 2.4 illustrates the proposed quantization architecture applied to the HiFiC model. The quantization process is designed to operate in a post-training manner, eliminating the need for retraining and enabling rapid deployment on the target hardware.

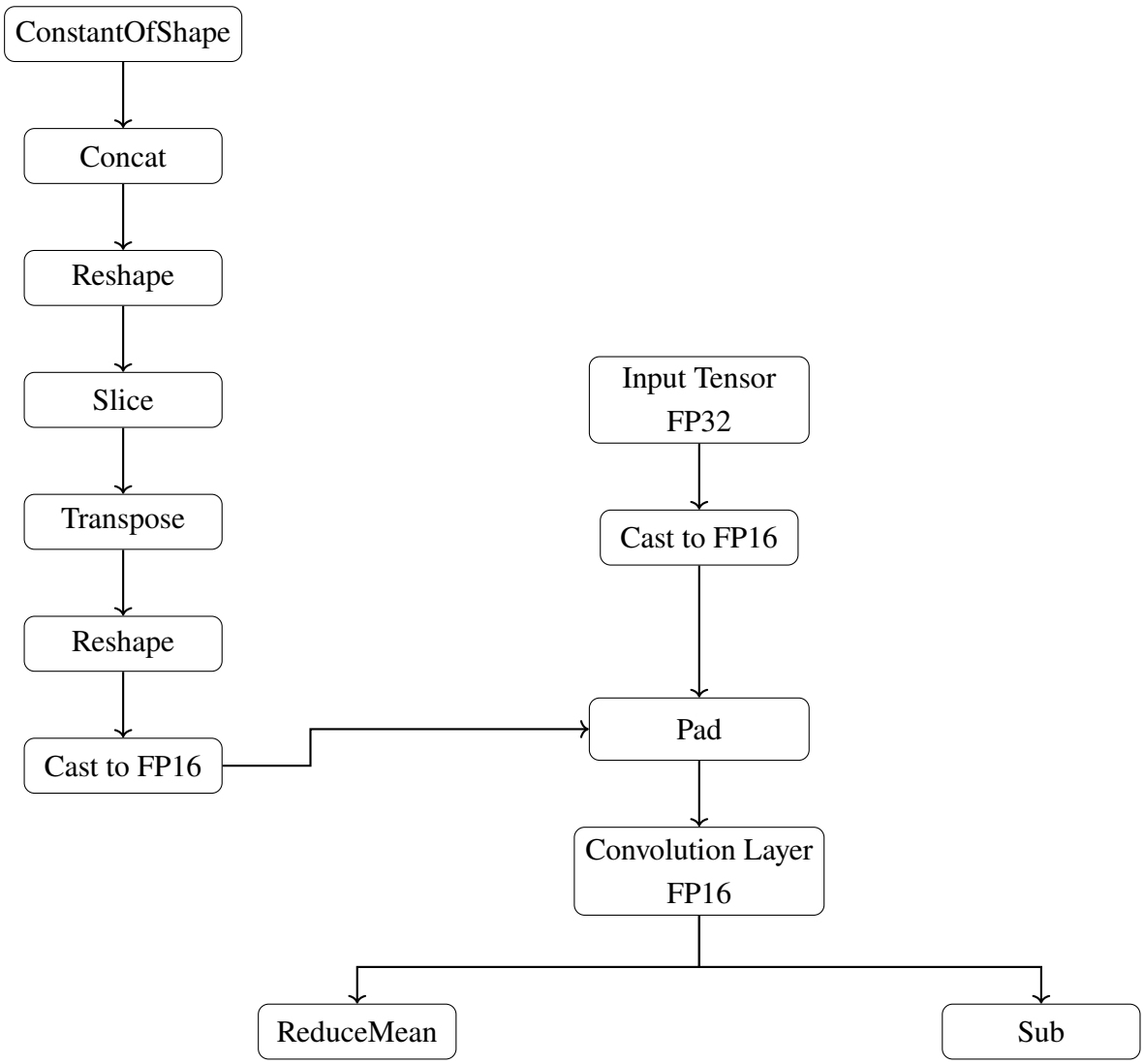

Figure 2.4 Mixed-precision quantization architecture applied to convolutional layers

2.3.1 Quantization Strategy

A post-training mixed-precision quantization approach is employed, where the internal computation of the neural network is performed using 16-bit floating-point (FP16) precision, while maintaining 32-bit floating-point (FP32) precision at the input and output interfaces. This strategy balances computational efficiency with numerical stability and compatibility with existing preprocessing and postprocessing pipelines.

The quantization strategy consists of the following key principles:

- **Interface Preservation:** The input and output tensors of the HiFiC model remain in FP32 format to ensure compatibility with external modules.
- **Internal Precision Reduction:** All intermediate feature maps and arithmetic operations within the network are executed in FP16 precision.

- **Static Weight Casting:** Convolutional kernels, bias terms, and learned parameters are statically converted from FP32 to FP16 to reduce memory footprint.

To enforce this precision boundary, explicit casting operations are inserted immediately after the input layer (FP32 to FP16) and before the output layer (FP16 to FP32). Redundant casting operators introduced during automatic model conversion are identified and removed to maintain a continuous FP16 dataflow throughout the network.

2.3.2 Operator-Level Precision Mapping

Following quantization, the HiFiC model executes the majority of its operations within the FP16 domain. These operations can be categorized into two main groups:

- **Computational Operations:** Convolution (Conv), transposed convolution (ConvTranspose), and matrix multiplication layers dominate the arithmetic workload of the Generator and are executed entirely in FP16 precision. This enables efficient utilization of FPGA DSP resources for multiply-accumulate (MAC) operations.
- **Non-linear and Element-wise Operations:** Activation functions (ReLU), element-wise addition, subtraction, multiplication, division, and exponential functions are also performed in FP16 precision, ensuring consistency across the computation graph.

By enforcing FP16 computation across all internal operators, the model achieves a significant reduction in memory bandwidth consumption and computational complexity, while preserving the structural integrity of the HiFiC architecture.

2.3.3 Quantization Rationale

The choice of FP16 precision is motivated by its favorable trade-off between numerical range and hardware efficiency. Compared to integer-based quantization schemes, FP16 offers higher dynamic range and improved stability for GAN-based architectures, where small numerical perturbations can otherwise lead to visible degradation in reconstructed images.

This mixed-precision quantization framework establishes a robust foundation for hardware acceleration of the HiFiC model on FPGA platforms, enabling efficient execution without compromising compression fidelity or perceptual quality.

2.4 Analysis of Computational Bottlenecks and Hardware Acceleration Priority

Although the HiFiC GAN architecture provides high perceptual quality for learned image compression, its inference pipeline introduces significant computational complex-

ity, particularly within the generator network. In order to design an efficient hardware–software co-design solution, it is essential to identify the main computational bottlenecks and prioritize the components that can benefit most from hardware acceleration.

To achieve this, a detailed runtime profiling was conducted on the generator network executed on a CPU platform. The experiment was performed on an Intel Core i9 processor operating at 2.2 GHz, and the execution time of each functional block within the generator was measured. The profiling results are summarized in Table 2.1.

Table 2.1 Runtime breakdown of HiFiC generator blocks executed on Intel Core i9 CPU @ 2.2 GHz

Block Name	**Execution Time (s)**	**Percentage (%)**
conv_block_init	23.139	2.16
res_block_0	111.327	10.37
res_block_1	102.859	9.58
res_block_2	97.376	9.07
res_block_3	96.871	9.03
res_block_4	96.799	9.02
res_block_5	102.545	9.55
res_block_6	106.396	9.91
res_block_7	113.640	10.59
res_block_8	118.866	11.08
upconv_blocks (Total 1–4)	87.696	8.17
conv_block_out	15.588	1.45
Others (Add, Overhead)	0.088	0.03
Total Runtime	**1073.19**	**100**

As shown in Table 2.1, the residual blocks clearly dominate the overall execution time of the generator network. The eight residual blocks together account for more than 78% of the total runtime, with each individual block contributing approximately 9–11%. In contrast, other components such as the initial convolution block, up-convolution blocks, and output convolution block contribute a significantly smaller portion of the total execution time.

The high computational cost of the residual blocks originates from their internal structure, which includes multiple convolutional layers, normalization operations, activation functions, and skip connections. These operations involve a large number of multiply–accumulate (MAC) computations and exhibit regular data access patterns with high data reuse. Such characteristics make residual blocks particularly well suited for

parallel and pipelined hardware implementations on FPGA. In addition to their dominant runtime contribution, all ResBlocks in the HiFiC GAN Generator share an identical computational configuration and tensor sizing. In particular, the input tensor of each 3×3 convolution inside a ResBlock has a fixed size of $18 \times 18 \times 960$, and the corresponding convolution output has a size of $16 \times 16 \times 960$. This spatial reduction is consistent with a 3×3 convolution with stride 1 under valid (no-padding) boundary handling, while the channel depth remains constant at 960.

Since every ResBlock applies the same sequence of operations (convolution–normalization–activation with a residual addition) on tensors of the same dimensions, the multiple ResBlocks represent repeated executions of the same compute kernel on different feature data rather than heterogeneous workloads.

Based on this analysis, the residual block is selected as the primary target for hardware acceleration in this work. By accelerating the residual blocks on FPGA, a substantial reduction in overall inference latency can be achieved while keeping the hardware design scope focused and manageable. The remaining components of the HiFiC GAN pipeline are executed in software, preserving flexibility and simplifying system integration.

This selective hardware acceleration strategy provides a balanced trade-off between performance improvement and design complexity, and establishes a scalable foundation for future extensions of the accelerator to additional components if required.

2.5 Implementation using high-level synthesis by Vitis High level systhesis

In this section, we will move on to the deployment process of the previously designed algorithm. We will program to create an IP block capable of accurately executing the mentioned algorithms. In this process, we will use the C++ programming language and the High-Level Synthesis (HLS) 2.5 library to generate an IP block implemented in VHDL.

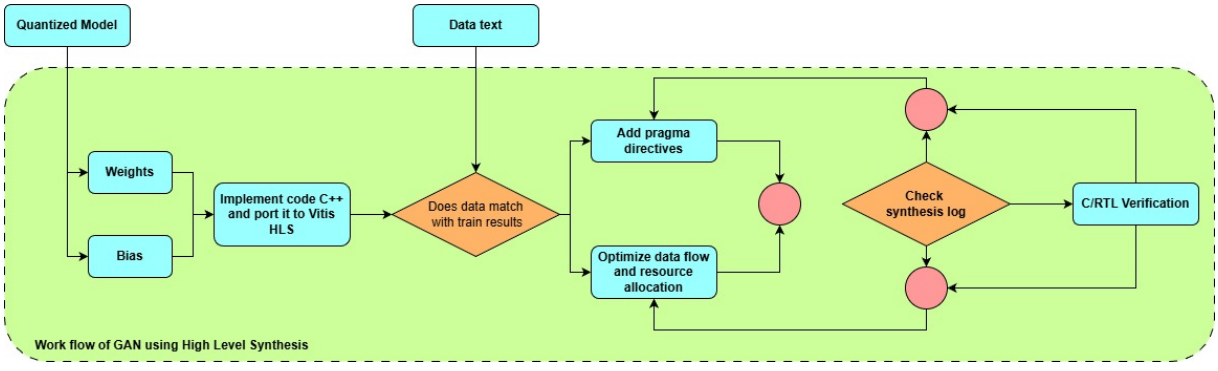

Figure 2.5 Workflow of Convolutional Neural Network using High Level Synthesis

2.5.1 Techniques used in the model

2.5.1.1 Linebuffer in Image Processing

In the proposed HiFiC GAN accelerator, line buffers are employed to enable efficient streaming execution of the 3×3 convolution layers inside the Generator Residual Blocks (ResBlocks). Since convolution kernels operate on spatially adjacent feature values, repeatedly fetching data from external memory would result in excessive memory bandwidth consumption. To address this issue, a two-line buffer architecture is adopted to maximize on-chip data reuse.

Figure 2.6 illustrates the *structural organization* of the two-line buffer. The input feature map is streamed in raster-scan order (row by row). At any given time, two previously processed rows are stored in the line buffer, while the current row is provided directly from the input stream. Together, these three rows are sufficient to construct the sliding 3×3 convolution window required for ResBlock computation.

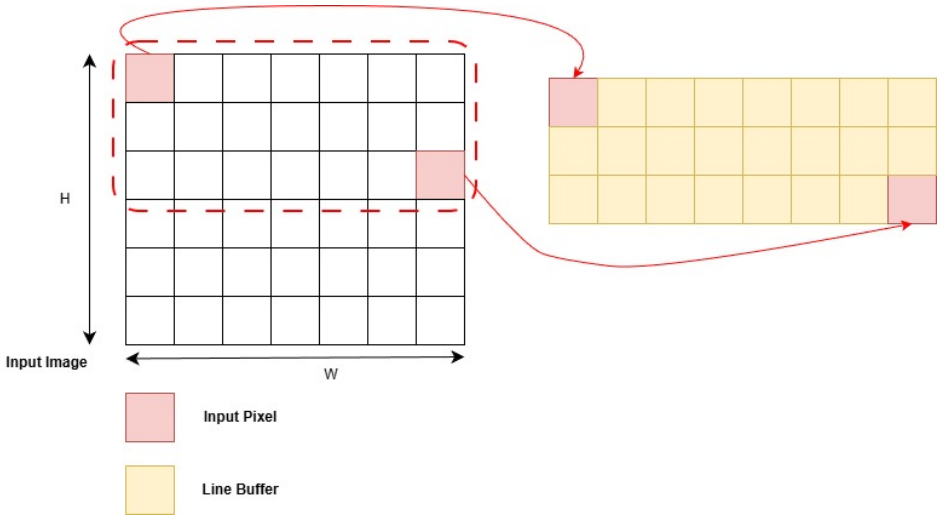

Figure 2.6 Two-line buffer structure for streaming convolution

Each incoming feature value is written into the second row of the line buffer at the corresponding column index. For example, if the current input feature corresponds to column j, it is stored at position $(\text{row } i-1, j)$ in the buffer. The first buffer row holds the feature values from row $i-2$, while the current input stream provides the values from row i.

Figure 2.7 further explains the *temporal behavior* of the line buffer during streaming execution. Before a new input value is accepted, the contents of the line buffer at the current column are shifted upward: data in the second buffer row are moved to the first row, and the newly arriving feature value is written into the second row. This vertical shift operation preserves the correct spatial alignment between neighboring rows.

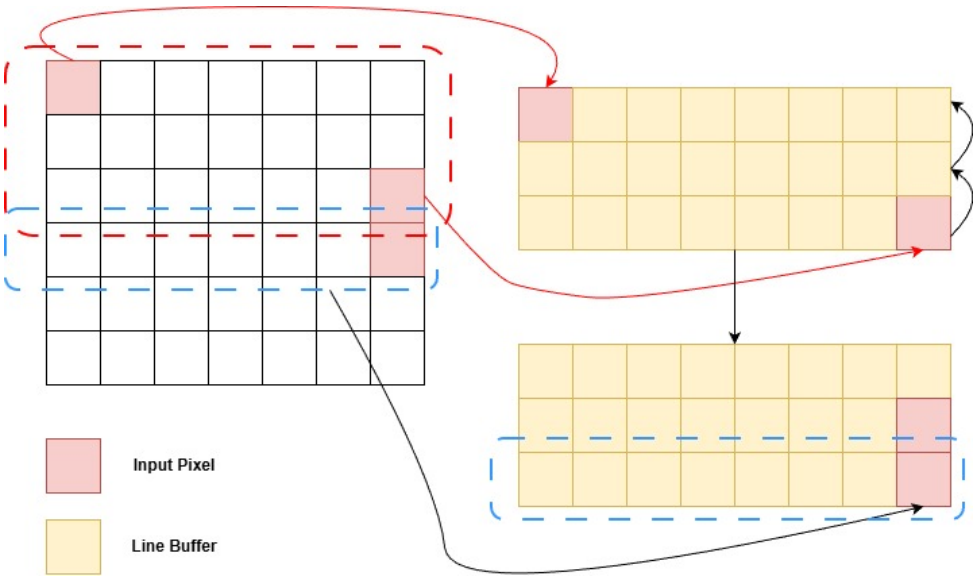

Figure 2.7 Two-line buffer dataflow and update mechanism

As the input stream advances horizontally across the feature map, this update process is repeated for each column. Once sufficient data are available, the convolution window can slide horizontally using values reused from the line buffer, requiring only one new input per clock cycle. This enables a fully pipelined execution, where one convolution result can be produced per cycle after pipeline initialization.

The two-line buffer architecture significantly reduces external memory access by keeping frequently reused feature values on-chip. This design minimizes latency, improves throughput, and is particularly well suited for FPGA-based acceleration, where memory bandwidth is limited and efficient data organization is critical for achieving high performance.

2.5.1.2 Window in Image Processing

The window shown in Fig. 2.8 is a data structure used to extract local regions of a feature map during convolution processing. In image and feature map processing, sliding windows are commonly employed to apply convolution and pooling operations over spatially adjacent data points.

In a conventional convolution implementation, the window moves both horizontally and vertically across the image with a fixed stride, sequentially generating one convolution output at a time. The window size is determined by the convolution kernel dimensions and can be adjusted according to the requirements of the target algorithm.

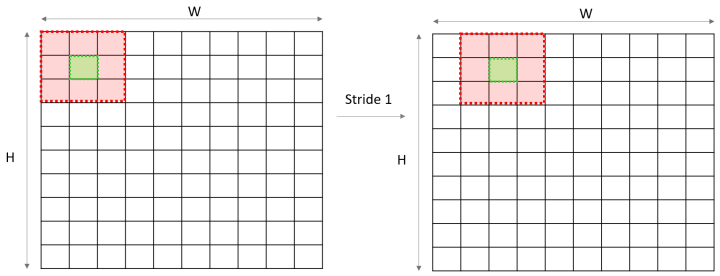

Figure 2.8 Sliding window structure for convolution processing

In the proposed ResBlock accelerator design, the window update mechanism is modified to support parallel computation. Instead of sliding the window horizontally across the feature map, multiple convolution windows are generated and processed in parallel along the horizontal direction. As a result, horizontal window movement is eliminated at the architectural level.

With this parallel window generation strategy, the window update operation is only performed in the vertical direction. When processing proceeds to the next row of the feature map, window data are updated using values from the line buffer and the incoming input stream. The spatial relationship between neighboring windows along the horizontal axis is preserved by parallel processing elements, each responsible for a fixed window position.

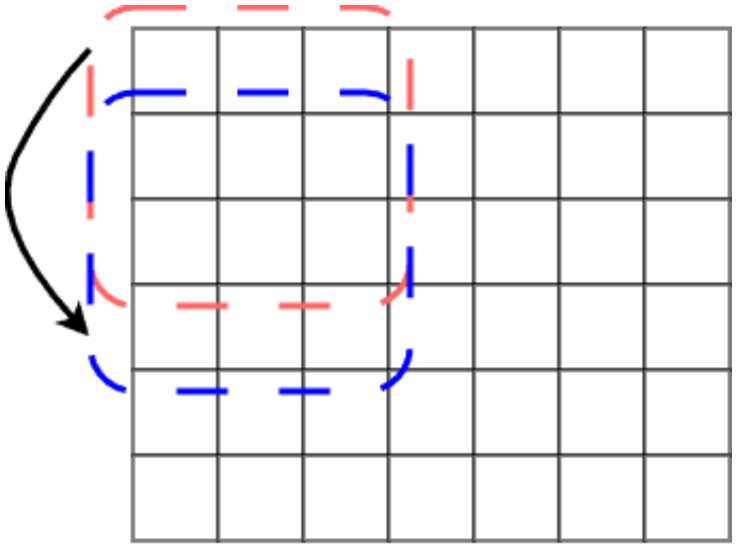

Figure 2.9 Parallel window update mechanism with vertical-only movement

This approach allows multiple convolution outputs to be computed simultaneously in each clock cycle while maintaining a fully streaming dataflow. By restricting window movement to the vertical direction, the design achieves higher throughput and reduced control complexity compared to traditional sequential sliding window implementations,

making it well suited for FPGA-based acceleration of ResBlock convolution layers.

2.5.1.3 Padding

Padding shown in Fig. 2.10 is a technique used to extend feature map boundaries in order to preserve spatial dimensions during convolution operations. Unlike conventional zero padding, the proposed HiFiC GAN model employs *reflection padding* to reduce boundary artifacts and maintain visual consistency near image edges.

In reflection padding, values outside the original feature map are generated by mirroring the interior feature values across the boundary. As illustrated in Fig. 2.10, edge pixels are reflected to form the padded region instead of being replaced by zeros. This padding strategy preserves local continuity and prevents abrupt intensity changes at the borders.

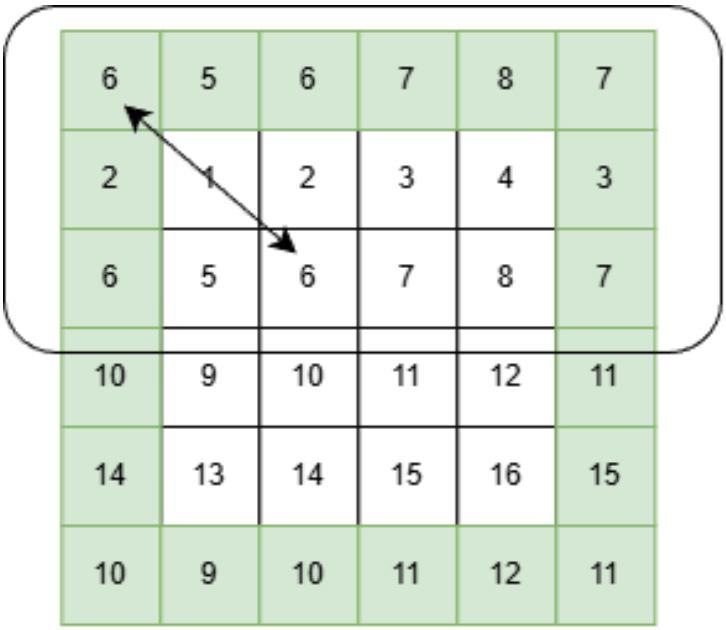

Figure 2.10 Reflection padding used in HiFiC GAN

In the HiFiC GAN Generator, reflection padding is applied before each 3×3 convolution inside the Residual Blocks. This ensures that the spatial resolution of feature maps remains unchanged while avoiding artificial edges introduced by zero padding. Reflection padding is particularly important for learned image compression models, where boundary artifacts can significantly degrade perceptual quality.

By preserving edge continuity and reducing convolution-induced border distortion, reflection padding contributes to improved reconstruction quality and more stable inference behavior in the Generator network. This padding method is therefore well suited for

FPGA-based acceleration of GAN models targeting high-fidelity image reconstruction.

2.6 Processing Element (PE) Architecture

To support high-throughput convolution in the ResBlock accelerator, a dedicated Processing Element (PE) is designed as the basic computation unit. Each PE is responsible for computing one partial convolution result using a set of input feature values and corresponding kernel weights.

Figure 2.11 illustrates the internal architecture of the proposed PE. The PE consists of multiple parallel multiply units, where each input feature map value (IFM) is multiplied with its corresponding weight. These multiplication operations are mapped directly onto FPGA DSP blocks to efficiently support FP16 arithmetic.

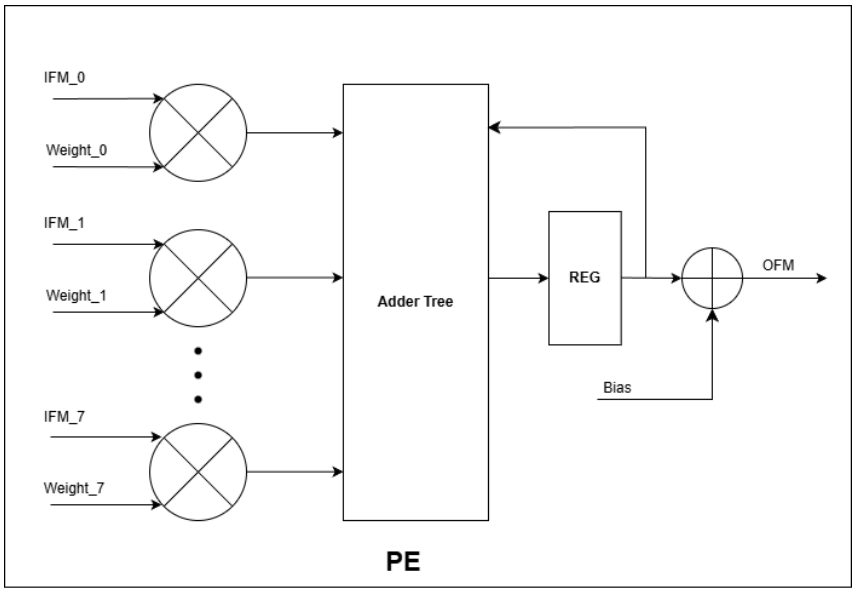

Figure 2.11 Processing Element (PE) architecture for convolution computation

The outputs of the multipliers are combined using an adder tree structure, which performs parallel accumulation to reduce the overall summation latency. A pipeline register is inserted after the adder tree to support high-frequency operation and continuous streaming. The accumulated result is then added with a bias term to produce the output feature map (OFM) value.

In the proposed design, multiple PEs operate in parallel to process multiple convolution windows simultaneously. This PE-level parallelism replaces traditional horizontal window sliding and enables the accelerator to achieve high throughput with a fully streaming dataflow. The modular PE architecture also allows scalable design by adjusting the number of instantiated PEs based on available FPGA resources.

2.7 ResBlock Hardware Architecture

This section presents the hardware architecture of the proposed ResBlock accelerator for the HiFiC GAN Generator. The design follows a streaming dataflow and hardware–software co-design approach, where the Processing System (PS) manages control and data movement, while the Programmable Logic (PL) performs compute-intensive operations.

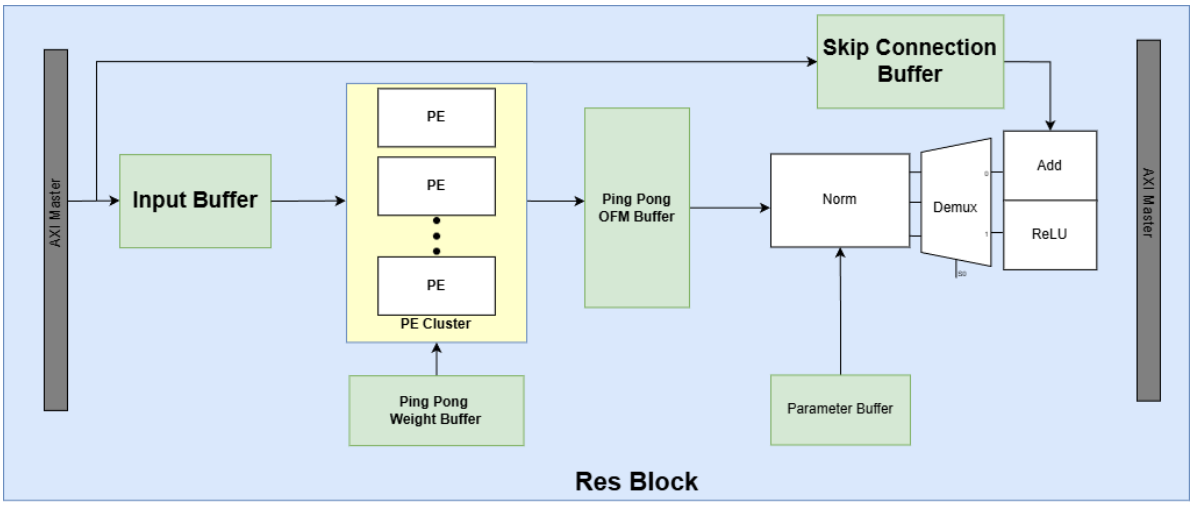

Figure 2.12 Proposed hardware architecture of the ResBlock accelerator

Figure 2.12 provides an overview of the ResBlock accelerator architecture. The input feature map is buffered and processed by a PE Cluster for parallel convolution computation, while skip connection buffering, normalization, and activation are integrated to support the residual structure of the ResBlock. The detailed dataflow and execution behavior of each component are analyzed in the following subsections.

2.7.1 From Line Buffer to Parallel Window Processing

As described in the Line Buffer and Window sections, a two-line buffer is sufficient for constructing a 3×3 convolution window under raster-scan streaming, because two previous rows can be stored on-chip while the current row is provided directly from the input stream. In conventional implementations, the window slides both horizontally and vertically across the feature map.

However, in the proposed design, **horizontal window movement is replaced by parallel window generation**. Specifically, the architecture instantiates a **PE Cluster** containing multiple Processing Elements (PEs) such that **all valid convolution windows on the same feature-map row are processed in parallel**. Therefore, within a row, each PE is assigned to a fixed horizontal window position (or a fixed output lane), and the update operation only needs to advance **downward to the next row**.

This design choice is enabled by the line buffer architecture, which continuously supplies spatially aligned feature data for multiple window positions without requiring repeated accesses to external memory.

2.7.2 PE Cluster and Convolution Throughput

The PE Cluster forms the compute core of the ResBlock accelerator. Each PE implements a pipelined multiply–accumulate (MAC) datapath for convolution, consisting of parallel multipliers, an adder tree, and pipeline registers. The multipliers are mapped to FPGA DSP resources to efficiently support FP16 arithmetic.

By replicating PEs across the horizontal dimension, the cluster computes multiple output pixels simultaneously for each processed feature-map row. Compared with sequential sliding-window implementations, this approach significantly increases throughput and simplifies control logic, at the cost of additional DSP usage. This trade-off is justified by the dominant runtime contribution of ResBlocks in the HiFiC GAN Generator.

To sustain continuous streaming execution, the PE Cluster is supported by a **ping-pong weight buffer**. While one weight bank feeds the current convolution computation, the other bank preloads the next set of weights, thereby preventing pipeline stalls during kernel switching.

2.7.3 OFM Buffering and Post-processing Pipeline

The output feature maps generated by the PE Cluster are temporarily stored in a **ping-pong OFM buffer**. This buffer decouples the convolution computation stage from subsequent post-processing stages, allowing the PE Cluster to continue operating while normalization and activation are applied to previously computed outputs.

The normalization unit applies scale and bias parameters stored in the **parameter buffer**, supporting the normalization operations required by the Generator network. After normalization, the data are forwarded to the residual addition and activation stages.

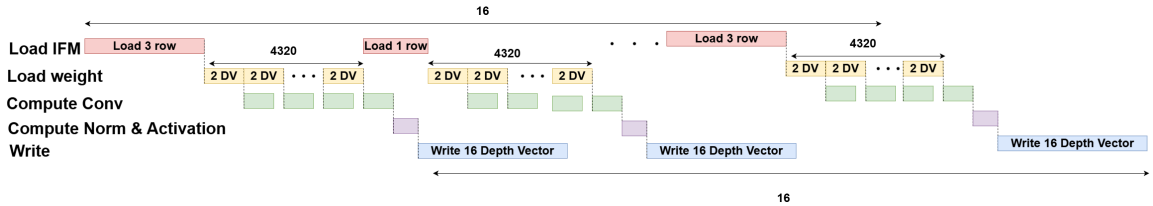

Figure 2.13 Pipeline timing diagram of the proposed ResBlock accelerator

Figure 2.13 illustrates the timing behavior of the proposed ResBlock accelerator under streaming execution. The pipeline begins with an initial warm-up phase, during

which multiple feature-map rows are loaded into the line buffer to construct valid 3×3 convolution windows.

Once the line buffer is fully populated, the PE Cluster starts parallel convolution processing across the entire feature-map row. During steady-state operation, convolution computation, output buffering, and write-back operations overlap in time. The use of ping-pong buffering for both weights and output feature maps enables continuous execution without pipeline stalls. While one OFM buffer bank is written with newly computed results, the other bank can be read by post-processing stages or prepared for the next write operation.

As a result, after the initial warm-up latency, the accelerator achieves a sustained throughput in which a full output feature vector is produced and written back in each pipeline iteration.

2.7.4 Skip Connection Buffer

A key characteristic of the ResBlock is the residual (skip) connection, which adds the original input feature map to the transformed output before applying the activation function. Since the main convolution path introduces latency due to pipelining and buffering, the input feature map must be delayed to align with the corresponding output feature map.

Therefore, a dedicated **skip connection buffer** is introduced to store and synchronize the input stream. After normalization, a demultiplexer selects the appropriate data path, and residual addition is performed using the synchronized skip data. A ReLU activation is then applied to produce the final ResBlock output feature map.

2.7.5 Summary

Overall, the proposed ResBlock accelerator architecture combines:

- **Line buffers** for spatial data reuse,
- **Parallel window processing** across each row via a PE Cluster,
- **Ping-pong buffering** for weights and OFM to maintain streaming throughput, and
- **A skip connection buffer** to satisfy residual addition requirements.

This integrated design enables high-throughput FPGA acceleration tailored to the computational characteristics of HiFiC GAN ResBlocks.

2.8 Application of ResBlock Accelerator

2.8.1 ResBlock Computational Structure

The Generator network in the HiFiC model is composed of multiple identical residual blocks (ResBlocks), each of which serves as the fundamental computational unit for feature transformation. Unlike generic Convolutional Neural Network (CNN) pipelines that decompose computation into independent operators, the ResBlock integrates several operations into a tightly coupled execution flow optimized for hardware implementation.

Figure 2.14 illustrates the computational structure of a single ResBlock used in this work.

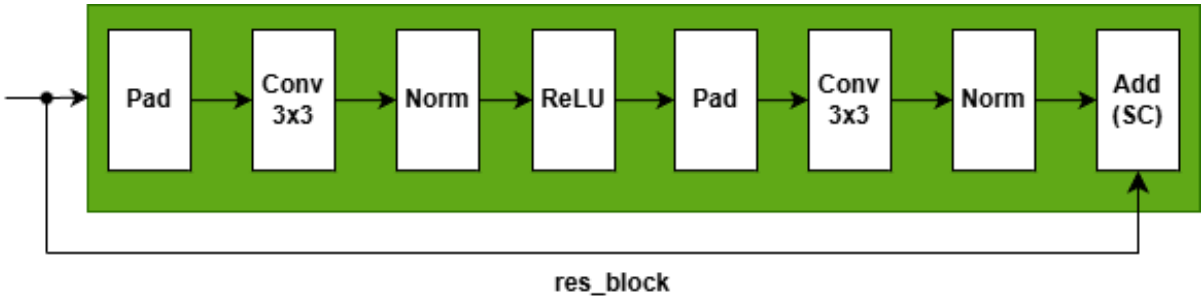

Figure 2.14 Computational flow of a ResBlock in the HiFiC Generator

As shown in Fig. 2.14, the ResBlock consists of two consecutive convolutional stages with a residual skip connection. The detailed computation flow is summarized as follows:

- **Padding:** Reflection-based padding is applied to the input feature map to preserve spatial dimensions when performing 3×3 convolutions. Padding is handled implicitly within the convolution engine to avoid additional memory accesses.
- **First Convolution and Normalization:** The padded input feature map is processed by a 3×3 convolution layer, followed by channel-wise normalization. This stage extracts local spatial features while stabilizing activation distributions across channels.
- **Activation Function:** A Rectified Linear Unit (ReLU) is applied after the first normalization stage. Notably, only a single non-linear activation is used within each ResBlock, which helps maintain signal fidelity in the generator network.
- **Second Convolution and Normalization:** The activated feature map undergoes a second 3×3 convolution and normalization stage. No activation function is applied after this stage.

- **Residual Addition:** The output of the second normalization stage is added element-wise to the original input feature map through a skip connection, producing the final ResBlock output.

Mathematically, the ResBlock operation can be expressed as:

$$\mathbf{Y} = \mathrm{Norm}_2\big(\mathrm{Conv}_2(\mathrm{ReLU}(\mathrm{Norm}_1(\mathrm{Conv}_1(\mathbf{X}))))\big) + \mathbf{X}$$

where $\mathbf{X}$ and $\mathbf{Y}$ denote the input and output feature maps of the ResBlock, respectively.

This ResBlock design is particularly suitable for GAN generators, as the linear output after residual addition avoids excessive non-linearity accumulation while preserving expressive feature transformations. Since all ResBlocks in the Generator share identical tensor dimensions and computational structure, a single hardware architecture can be reused across multiple ResBlock instances.

2.8.2 Top-Level Kernel Interface and Execution Model

The ResBlock accelerator is exposed to the system as a standalone hardware kernel, enabling seamless integration into the hardware–software co-design flow. A single top-level function is synthesized as an IP core and invoked repeatedly by the Processing System (PS) to process multiple ResBlocks in the Generator.

2.8.2.1 Kernel Interface.

The top-level kernel interfaces directly with external memory through AXI4 master ports. Separate memory interfaces are used for input/output feature maps, convolution weights, bias terms, and normalization parameters. This separation enables concurrent memory transactions and reduces access contention during execution.

All tensor dimensions are fixed at compile time, allowing static memory allocation and simplified address generation. The kernel interface can be abstracted as follows:

- **Input feature map X** stored in external DDR memory
- **Output feature map Y** written back to external DDR memory
- **Convolution parameters** (weights and biases) for both convolution stages
- **Normalization parameters** (γ, β, and ε)

A lightweight AXI-Lite control interface is used to configure scalar parameters and trigger kernel execution.

2.8.2.2 *Execution Model.*

The ResBlock accelerator operates on a complete feature map per invocation. For each call, the kernel performs the full ResBlock computation, including two convolution stages, normalization, activation, and residual addition.

At a higher level, the execution of the Generator can be represented by the following pseudo-code:

```
for each ResBlock in Generator:
    load input feature map X from DDR
    load ResBlock parameters (W1, B1, W2, B2, , )
    Y = ResBlock_Accelerator(X, parameters)
    X = Y
```

This execution model treats the ResBlock as an atomic hardware operation. Intermediate results are retained on-chip whenever possible, while only the final output feature map is written back to external memory.

2.8.2.3 *Design Implications.*

By encapsulating the ResBlock as a single kernel invocation, the design avoids fine-grained synchronization between individual CNN operators. This approach reduces control overhead and allows the accelerator to focus on maximizing throughput through data reuse, on-chip buffering, and pipeline parallelism.

The following sections detail the internal architecture of the ResBlock accelerator, including buffer organization, parallel processing elements, and execution scheduling.

2.8.3 Tensor Abstraction and Parameterization

To bridge high-level tensor operations and low-level hardware implementation, the ResBlock accelerator employs a lightweight tensor abstraction layer. This abstraction provides a uniform representation of multi-dimensional data while preserving compatibility with static memory allocation and HLS synthesis constraints.

2.8.3.1 *Tensor Shape Representation*

Each tensor is described by a fixed shape defined at compile time, consisting of batch size, spatial dimensions, and channel depth. In this design, all ResBlocks operate on identical tensor shapes, eliminating the need for dynamic shape handling.

Formally, a tensor shape is represented as:

$$\text{Shape} = (N, H, W, C)$$

where N denotes the batch size, H and W denote spatial height and width, and C denotes the number of channels.

For the HiFiC Generator, the following fixed configuration is used:

- $N = 1$
- $H = W = 16$
- $C = 960$

This fixed-shape assumption simplifies address generation and enables aggressive compile-time optimization.

2.8.3.2 Tensor Memory Abstraction

A lightweight tensor memory wrapper is introduced to encapsulate raw memory pointers together with shape metadata. Rather than relying on dynamic memory allocation, this wrapper provides deterministic indexing into flattened memory layouts compatible with AXI-based external memory.

Each tensor is stored in a contiguous one-dimensional memory layout using a row-major order, defined as:

$$\text{index}(n,h,w,c) = ((n \cdot H + h) \cdot W + w) \cdot C + c$$

This mapping allows direct computation of memory addresses using compile-time constants, avoiding costly address translation logic at runtime.

2.8.4 Hardware-Friendly Design Considerations

The tensor abstraction is designed with hardware synthesis constraints in mind:

- All tensor dimensions are compile-time constants, enabling static loop bounds and predictable access patterns.
- No dynamic memory allocation is used in hardware; all buffers are either statically allocated on-chip or mapped to external memory.
- The abstraction layer introduces no additional control logic and is fully inlined during HLS synthesis.

By separating tensor indexing logic from computation logic, the design improves code readability while maintaining a clear mapping between high-level tensor semantics and low-level hardware operations.

2.8.4.1 Role in the Overall Architecture

The tensor abstraction layer serves as a foundation for subsequent architectural components, including on-chip buffering, convolution engines, and pipeline scheduling. By standardizing tensor access patterns, it enables consistent reuse of hardware modules across multiple ResBlock invocations without modification.

The next section details how tensors are buffered and reused within the accelerator to maximize memory efficiency and throughput.

2.8.5 On-Chip Buffering and Memory Hierarchy

Efficient memory organization is critical to achieving high throughput in the ResBlock accelerator, as convolutional operations exhibit substantial data reuse across both spatial and channel dimensions. To minimize external memory traffic and sustain pipeline execution, the proposed design adopts a hierarchical on-chip buffering strategy tailored specifically to the fixed ResBlock computation pattern.

2.8.5.1 Overview of Buffer Organization

The accelerator employs multiple on-chip buffers, each serving a distinct role in the ResBlock execution:

- rolling buffering of input feature maps for sliding window access,
- buffering of convolution weights for reuse across spatial positions,
- residual buffering for skip connections,
- caching of intermediate convolution results to avoid redundant computation.

All buffers are statically allocated and explicitly mapped to on-chip memory resources to ensure predictable access latency and bandwidth.

2.8.5.2 Buffer-Level Execution Flow

The interaction between on-chip buffers during ResBlock execution can be summarized by the following pseudo-code, which emphasizes data movement and reuse rather than arithmetic computation:

```
initialize input_row_buffer
initialize weight_buffer
initialize residual_buffer
initialize conv_output_cache
```

```
for each spatial row r:
    load next input row from DDR into input_row_buffer

    if first pass:
        store input row into residual_buffer

    for each spatial column w:
        for each output channel block:
            compute convolution using input_row_buffer and weight_
            store result into conv_output_cache
            accumulate statistics for normalization

for each spatial row r:
    for each spatial column w:
        for each output channel:
            read cached convolution result
            apply normalization and activation
            add residual from residual_buffer
            write output feature to memory
```

This pseudo-code highlights that input features and weights are reused across multiple convolution windows, while convolution results are cached on-chip to support normalization without recomputation.

2.8.5.3 Input Feature Buffer

Input feature maps are buffered using a rolling row-buffer mechanism that retains multiple rows of the feature map on-chip. This buffer enables continuous generation of 3×3 convolution windows as computation slides across the spatial dimension.

Specifically, three row blocks of the input feature map are maintained simultaneously, allowing the convolution engine to access all required neighboring pixels without reloading data from external memory.

2.8.5.4 Weight Buffer

Convolution weights are stored in a dedicated on-chip buffer prior to computation. Since the same convolution kernels are applied across all spatial locations, weight parameters are heavily reused during ResBlock execution, significantly reducing external memory bandwidth requirements.

2.8.5.5 *Residual Buffer*

To support the skip connection inherent in the ResBlock structure, the original input feature map is preserved in a residual buffer until the second convolution and normalization stage completes. This allows element-wise residual addition to be performed locally on-chip.

2.8.5.6 *Intermediate Result Cache*

Channel-wise normalization requires access to convolution outputs both for statistical analysis and for affine transformation. To prevent redundant convolution execution, intermediate outputs are cached on-chip after the first computation pass and reused during the second pass.

2.8.5.7 *Memory Resource Binding*

Each buffer is explicitly bound to an appropriate on-chip memory resource based on its size and access characteristics:

- block RAM (BRAM) is used for frequently accessed buffers such as input rows and weight storage,
- UltraRAM (URAM) is used for larger buffers with longer lifetimes, including residual storage and intermediate caches.

2.8.5.8 *Summary*

By explicitly structuring data movement around on-chip buffers, the ResBlock accelerator maximizes data reuse, minimizes external memory access, and enables sustained pipeline execution. This buffering strategy directly supports the parallel convolution engine described in the following section.

2.8.6 Parallel Convolution Engine

The core computation of the ResBlock accelerator is performed by a highly parallel convolution engine designed to exploit spatial and channel-level parallelism. Given the fixed tensor dimensions and regular computation pattern, the convolution engine adopts a processing-element (PE) based architecture with SIMD-style execution across channels.

2.8.6.1 *Processing Element Organization*

Each Processing Element (PE) is responsible for computing the convolution output for a single spatial position (h, w) across a block of output channels. Multiple PEs operate in parallel to process an entire row of output feature maps concurrently.

Within each PE, channel-wise parallelism is further exploited using SIMD lanes,

allowing multiple input channels to be processed simultaneously. Partial sums are accumulated through a pipelined reduction tree to produce final convolution results.

2.8.6.2 Parallel Execution Model

The parallel execution strategy of the convolution engine can be summarized by the following pseudo-code, annotated with HLS directives to indicate hardware parallelism:

```
#pragma HLS UNROLL factor=PE_UNROLL
for each PE in parallel:                // spatial parallelism
    initialize accumulation registers

    for each output channel block:
        #pragma HLS PIPELINE II=1
        for each input channel group:   // SIMD lanes
            #pragma HLS UNROLL
            multiply input features with weights
            accumulate partial sums

    write convolution results to local buffer
```

This execution model enables concurrent computation across spatial positions (via multiple PEs) and across channel dimensions (via SIMD lanes), significantly increasing throughput.

2.8.6.3 Pipeline and Latency Control

To sustain continuous execution, the innermost loops of the convolution engine are fully pipelined with an initiation interval (II) of one cycle. Floating-point arithmetic operators are explicitly bound to DSP resources and configured with fixed latency to ensure timing closure and predictable pipeline behavior.

2.8.6.4 HLS Pragmas and Their Roles

The following HLS pragmas are employed to realize the parallel convolution architecture:

- **`#pragma HLS UNROLL`**: Used to unroll loops over Processing Elements and SIMD lanes, enabling true hardware parallelism across spatial positions and channel groups.
- **`#pragma HLS PIPELINE II=1`**: Applied to the innermost accumulation loops to allow a new computation to be initiated every cycle, maximizing throughput.

- **`#pragma HLS ARRAY_PARTITION`**: Used to partition input feature buffers, weight buffers, and accumulation arrays, ensuring that parallel accesses can be served without memory access conflicts.
- **`#pragma HLS BIND_OP`**: Explicitly binds floating-point multiplication operations to DSP blocks and enforces fixed latency for addition operations, improving timing predictability and resource utilization.

2.8.6.5 Mapping to the Implemented Design

In the implemented accelerator, the number of PEs and SIMD lanes is selected based on available FPGA resources and target throughput. This parameterized design allows scalability by adjusting unroll factors and buffer partitioning without altering the overall architecture.

By combining PE-level spatial parallelism, SIMD-style channel parallelism, and aggressive loop pipelining, the convolution engine achieves high computational throughput while maintaining a compact and reusable hardware structure. This parallel execution model forms the computational backbone of the ResBlock accelerator.

2.8.7 Two-Pass Execution Strategy (Architecture-Level View)

Channel-wise normalization introduces a global dependency across the channel dimension: the mean and variance for a given spatial position must be derived from the full set of convolution outputs before the affine transform can be applied. To satisfy this dependency without re-executing convolution, the ResBlock accelerator adopts a two-pass execution strategy, illustrated in Fig. 2.15.

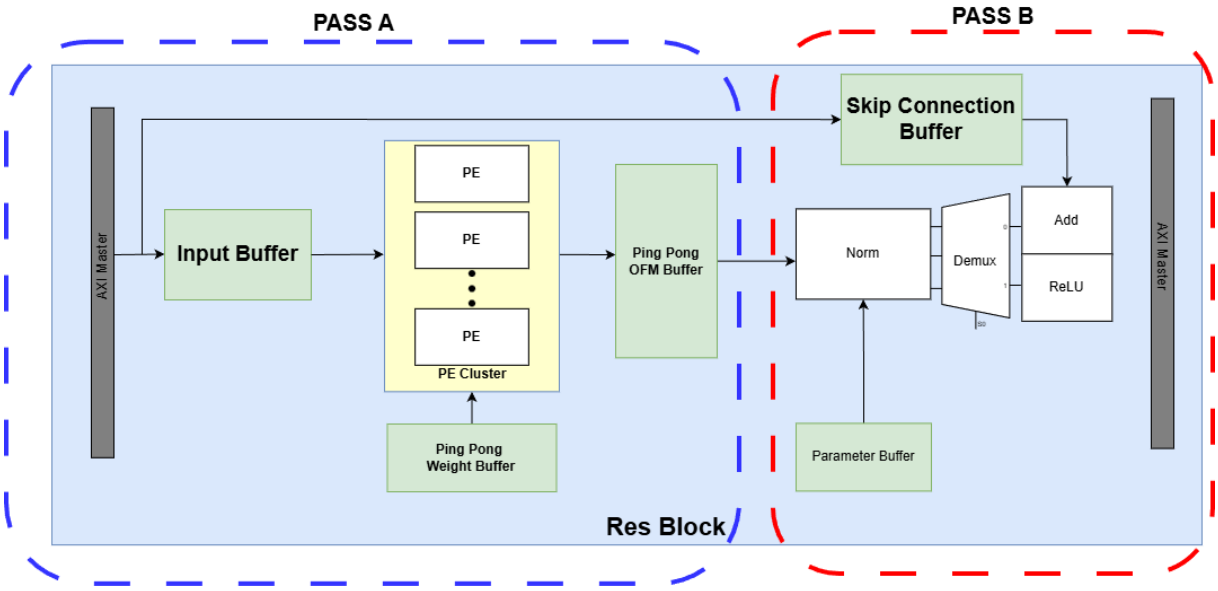

Figure 2.15 Two-pass execution architecture of the ResBlock accelerator.

The left (blue dashed) region performs convolution and produces buffered OFMs (Pass A). The right (red dashed) region performs normalization and conditional post-processing (Pass B), including ReLU (for the first stage) or residual addition (for the second stage).

2.8.7.1 Architectural partitioning in Fig. 2.15.

The diagram explicitly separates the ResBlock accelerator into two cooperating regions:

- **Pass A (blue dashed region): Convolution production path.** This region is responsible for generating convolution outputs using a PE cluster that operates with spatial parallelism. Each Processing Element (PE) computes the convolution for a distinct spatial window of the output feature map, allowing multiple convolution windows within the same row to be processed simultaneously. By streaming input data and reusing weights across these parallel windows, the PE cluster sustains continuous OFM production at high throughput.This window-level parallelism matches the fixed spatial dimensions of the ResBlock and allows the accelerator to exploit deterministic data reuse patterns.
- **Pass B (red dashed region): Normalization and output formation path.** This region consumes buffered OFMs, applies channel-wise normalization using stored parameters, and then performs conditional post-processing depending on whether the accelerator is executing the first or second stage of the ResBlock.

This partition is not only conceptual; it reflects the actual data dependency: normalization cannot complete until statistics for a spatial position are known, therefore convolution results must be buffered to decouple computation from post-processing.

2.8.7.2 Pass A: Convolution + OFM buffering (left side).

As shown in Fig. 2.15, Pass A begins at the **AXI Master** interface and **Input Buffer**, which stage input feature-map data to match the internal streaming rate. The key compute structure is the **PE Cluster**, which provides spatial parallelism (multiple PEs operating concurrently) while exploiting channel reuse by repeatedly applying the same weights across spatial positions.

Two buffering mechanisms support high-throughput convolution:

- **Ping-Pong Weight Buffer**: weights are prefetched and alternated between two banks so that the PE Cluster can compute while the next weight block is loaded, reducing stalls due to parameter access.

- **Ping-Pong OFM Buffer**: convolution outputs are written into an on-chip OFM buffer that decouples the PE Cluster from the downstream normalization stage. While one bank is being filled by Pass A, the other bank can be read by Pass B.

In addition, the design retains the original input in the **Skip Connection Buffer** (top path in Fig. 2.15). This buffer preserves the residual tensor locally so that residual addition can be performed without reloading the input from external memory.

2.8.7.3 Pass B: Normalization + conditional post-processing (right side).

Pass B reads OFM data from the OFM buffer and applies **Norm** using parameters supplied by the **Parameter Buffer** (e.g., γ, β, and ε). This stage transforms raw convolution outputs into normalized values ready for either activation (stage 1) or residual formation (stage 2).

After normalization, the diagram contains a **Demux** block, which represents a mode-select mechanism (controlled by the ResBlock stage, e.g., `times` in the implementation). The Demux selects one of two output formation paths:

- **Stage 1 (Conv1 path): Norm $\rightarrow$ ReLU**. The normalized result is passed through a single ReLU nonlinearity. This corresponds to the ResBlock design choice where **only one ReLU is applied per ResBlock**, placed after the first normalization stage.
- **Stage 2 (Conv2 path): Norm $\rightarrow$ Add (Skip Connection)**. The normalized output is combined with the preserved residual data from the **Skip Connection Buffer**. In this design, the ResBlock output is linear after the residual addition (i.e., no additional ReLU is required after Add), matching the computation graph used in the generator.

Finally, results are streamed back to external memory through the right-side **AXI Master** interface.

2.8.7.4 Why two-pass improves efficiency.

The two-pass strategy is advantageous because it avoids redundant compute and converts a global dependency into a buffered pipeline:

- **No convolution recomputation:** convolution results are produced once in Pass A and reused by Pass B.
- **Correct normalization:** statistics required for channel-wise normalization are computed consistently across the channel dimension before affine transform is applied.

- **Sustained throughput:** ping-pong buffers allow Pass A and Pass B to overlap in time, minimizing idle cycles due to memory or dependency stalls.
- **Clean separation of concerns:** Pass A is compute-heavy (PE Cluster), while Pass B is control/parameter-heavy (Norm + conditional logic), simplifying timing closure and resource mapping.

2.8.7.5 Architecture-Level Pseudo-Code of Two-Pass Execution

The following pseudo-code summarizes the two-pass execution strategy illustrated in Fig. 2.15, emphasizing dataflow, buffering, and control flow rather than arithmetic details:

```
# Pass A: Convolution production
for each output row r:
    stream input rows into Input Buffer
    store input features into Skip Connection Buffer

    parallel for each PE (spatial window):
        for each output channel block:
            compute convolution
            write OFM to Ping-Pong OFM Buffer
            accumulate statistics for normalization

# Pass B: Normalization and output formation
for each output row r:
    parallel for each PE (spatial window):
        for each output channel:
            read cached OFM from OFM Buffer
            apply normalization using stored parameters

            if stage == Conv1:
                apply ReLU
                write result as intermediate feature
            else if stage == Conv2:
                read residual from Skip Connection Buffer
                perform element-wise addition
                write final output feature
```

This pseudo-code reflects the architectural separation between convolution production (Pass A) and post-processing (Pass B), as well as the reuse of buffered data to

satisfy normalization dependencies without recomputing convolution results.

2.8.7.6 Summary.

Figure 2.15 captures the key architectural idea of the implemented accelerator: convolution is executed in a high-throughput producer stage (Pass A), while normalization and final output formation are executed in a consumer stage (Pass B). A Demux-controlled post-processing path ensures that a single hardware structure can support both ResBlock stages, applying **exactly one ReLU** after the first normalization and performing residual addition at the end of the block.

2.8.8 Sliding Row Scheduling and Pipeline Overlap

The pipeline timing behavior of the proposed ResBlock accelerator, previously illustrated in Figure 2.13, is revisited here from the perspective of sliding row scheduling and overlapped execution. While the figure was earlier used to explain output feature-map buffering behavior, it also reveals how data movement and computation are carefully scheduled to sustain continuous throughput.

2.8.8.1 Sliding Row Scheduling.

As shown in Figure 2.13, the accelerator processes the input feature map in a row-wise streaming manner rather than buffering the entire feature map on-chip. During the initial warm-up phase, multiple consecutive rows are loaded into the line buffer to construct valid 3×3 convolution windows. This warm-up corresponds to the "Load 3 row" stage in the timing diagram.

Once the pipeline enters steady-state operation, only one new input row is loaded per iteration. Previously buffered rows are reused to form convolution windows for the next output row, while obsolete rows are discarded. This sliding mechanism enables vertical traversal of the feature map using a fixed-size on-chip buffer, significantly reducing memory requirements.

2.8.8.2 Pipeline Overlap.

After the warm-up phase, Figure 2.13 shows that input loading, convolution computation, normalization, and output writing are overlapped in time. While the PE Cluster computes convolution results for the current row, the accelerator simultaneously:

- loads the next input row from external memory,
- performs normalization and post-processing for the previous row,
- writes finalized output vectors back to memory.

This overlap effectively hides memory access latency behind computation and prevents pipeline stalls, ensuring high utilization of the PE Cluster.

2.8.8.3 Steady-State Throughput.

In steady-state operation, each pipeline iteration produces one output row while preparing data for the next iteration. The use of ping-pong buffering for both weights and output feature maps enables concurrent read and write operations, further supporting uninterrupted execution.

The timing behavior depicted in Figure 2.13 highlights that once the pipeline is filled, the accelerator sustains a regular execution pattern in which load, compute, and write stages proceed concurrently across successive rows.

To bridge the architectural view and the actual hardware realization, the same two-pass pseudo-code is revisited below with explicit HLS directives. While the previous pseudo-code describes the logical separation of computation stages, the following version illustrates how these stages are scheduled, overlapped, and parallelized in the synthesized hardware.

2.8.8.4 Implementation-Level Pseudo-Code with HLS Pragmas

The same two-pass pseudo-code is rewritten below with HLS directives to illustrate how the architecture is realized as overlapped hardware execution.

```
# Pass A: Convolution production
for each output row r:
    #pragma HLS PIPELINE
    # (Overlaps row-level load/compute/store across iterations)

    stream input rows into Input Buffer
    store input features into Skip Connection Buffer

    #pragma HLS UNROLL factor=PE_UNROLL
    # (Spatial parallelism: multiple windows computed in parallel)
    parallel for each PE (spatial window):

        for each output channel block:
            #pragma HLS PIPELINE II=1
            # (Throughput: start a new MAC group every cycle)
            compute convolution
```

```
            write OFM to Ping-Pong OFM Buffer
            accumulate statistics for normalization

# Pass B: Normalization and output formation
for each output row r:

    #pragma HLS UNROLL factor=PE_UNROLL
    # (Spatial parallelism reused for post-processing)
    parallel for each PE (spatial window):

        #pragma HLS PIPELINE II=1
        # (Stream normalization/activation/add at 1 output per cyc
        for each output channel:
            read cached OFM from OFM Buffer
            apply normalization using stored parameters

            if stage == Conv1:
                apply ReLU
                write result as intermediate feature
            else if stage == Conv2:
                read residual from Skip Connection Buffer
                perform element-wise addition
                write final output feature
```

Note: In the actual HLS implementation, feature/weight buffers are partitioned (e.g., `#pragma HLS ARRAY_PARTITION`) to provide enough read/write ports for the unrolled PEs and SIMD lanes.

The inserted HLS directives in the pseudo-code correspond to the implementation choices in the ResBlock kernel:

- **`#pragma HLS PIPELINE` (row-level scheduling):** This directive pipelines the outer loop over output rows, enabling overlapped execution across successive iterations. In steady state, while the accelerator computes convolution for row r, it can concurrently load data for row $r+1$ and post-process/write back results for row $r-1$, matching the overlap shown in Figure 2.13.
- **`#pragma HLS UNROLL factor=PE_UNROLL` (spatial parallelism):** This directive unrolls the PE loop so that multiple Processing Elements are instantiated in hardware. Each PE corresponds to a distinct spatial window (e.g., different w

positions within a row), allowing multiple convolution windows to be processed simultaneously. This is consistent with the implementation loop `for (pe = 0; pe < PEs; pe++)` unrolled by `PE_UNROLL`.

- **`#pragma HLS PIPELINE II=1` (streaming throughput inside blocks):** This directive pipelines the inner loops so that a new computation step is initiated every cycle (initiation interval of 1). It is applied to both the convolution accumulation over channel groups and the post-processing loop over output channels, allowing normalization/activation/add to stream at a sustained rate.

- **`#pragma HLS ARRAY_PARTITION` (multi-port buffer access):** With PE unrolling and SIMD-style channel processing, multiple parallel reads/writes are required per cycle. Partitioning feature and weight buffers (e.g., cyclic partition by lane factor) increases effective memory ports, preventing access conflicts that would otherwise break the pipeline and reduce achieved II.

Overall, the architecture-level pseudo-code describes *what* the ResBlock accelerator computes, while the pragma-annotated pseudo-code describes *how* the same computation is scheduled, overlapped, and parallelized in hardware.

2.8.8.5 Impact on Overall Performance.

By combining sliding row scheduling with overlapped pipeline execution, the ResBlock accelerator achieves:

- reduced on-chip buffer requirements compared to full-frame buffering,
- improved throughput through effective latency hiding,
- predictable and deterministic execution well suited for FPGA implementation.

This scheduling strategy complements the parallel convolution engine and two-pass execution model, completing the end-to-end dataflow optimization of the ResBlock accelerator.

2.8.9 Summary of ResBlock Hardware Design

This section summarizes the key architectural decisions, trade-offs, and reusability aspects of the proposed ResBlock hardware accelerator developed for the HiFiC Generator.

2.8.9.1 Key Design Choices.

The ResBlock accelerator is designed around the observation that all ResBlocks in the HiFiC Generator share identical tensor dimensions and computational structure.

Instead of decomposing the network into fine-grained operators, the design encapsulates the entire ResBlock as a single hardware kernel. This approach enables tight integration of convolution, normalization, activation, and residual addition, minimizing intermediate memory traffic and control overhead.

A two-pass execution strategy is adopted to correctly support channel-wise normalization without recomputing convolution. Convolution results are produced once, buffered on-chip, and reused during normalization and post-processing. Spatial parallelism is exploited through a PE cluster, where multiple convolution windows within the same output row are processed concurrently. Sliding row scheduling and extensive pipelining allow input loading, computation, and write-back to overlap in time, sustaining high throughput after pipeline warm-up.

2.8.9.2 Design Trade-offs.

The primary trade-off in this design is reduced flexibility in exchange for higher performance and efficiency. By fixing tensor dimensions and specializing the accelerator for a single ResBlock configuration, the design avoids the overhead associated with fully programmable CNN accelerators. However, this specialization limits applicability to models with matching dimensions and computation patterns.

Another trade-off concerns on-chip memory usage. The use of multiple buffers (line buffers, ping-pong OFM buffers, skip connection buffers, and parameter buffers) increases BRAM and URAM consumption. This cost is accepted to eliminate redundant memory accesses and to enable pipeline overlap between computation stages, which is critical for achieving high utilization of the PE cluster.

2.8.9.3 Reusability and Scalability.

Despite being dimension-specific, the ResBlock accelerator is reusable across the entire Generator network because all ResBlocks share the same structure and tensor sizes. The Processing System can invoke the same hardware kernel repeatedly with different parameters, making the accelerator effectively reusable at the network level.

Furthermore, the architecture is scalable along several axes. The degree of spatial parallelism can be adjusted by modifying the number of instantiated PEs, while channel-level throughput can be tuned by changing SIMD width and unrolling factors. These parameters allow the design to be retargeted to different FPGA platforms or resource budgets without altering the fundamental execution model.

2.8.9.4 Overall Assessment.

In summary, the proposed ResBlock hardware design strikes a deliberate balance between specialization and reuse. By focusing on the dominant computational bottle-

neck of the HiFiC Generator, the accelerator achieves high throughput through architectural integration, buffering, and pipeline overlap, while remaining reusable across all ResBlocks in the network.

2.9 Integration into HW–SW Co-Design

Figure 2.16 illustrates the integration of the proposed ResBlock accelerator into the overall hardware–software co-design framework of the HiFiC inference system. The figure highlights the functional partitioning between software execution on the Processing System (PS) and hardware acceleration on the Programmable Logic (PL), as well as the data flow across system memory.

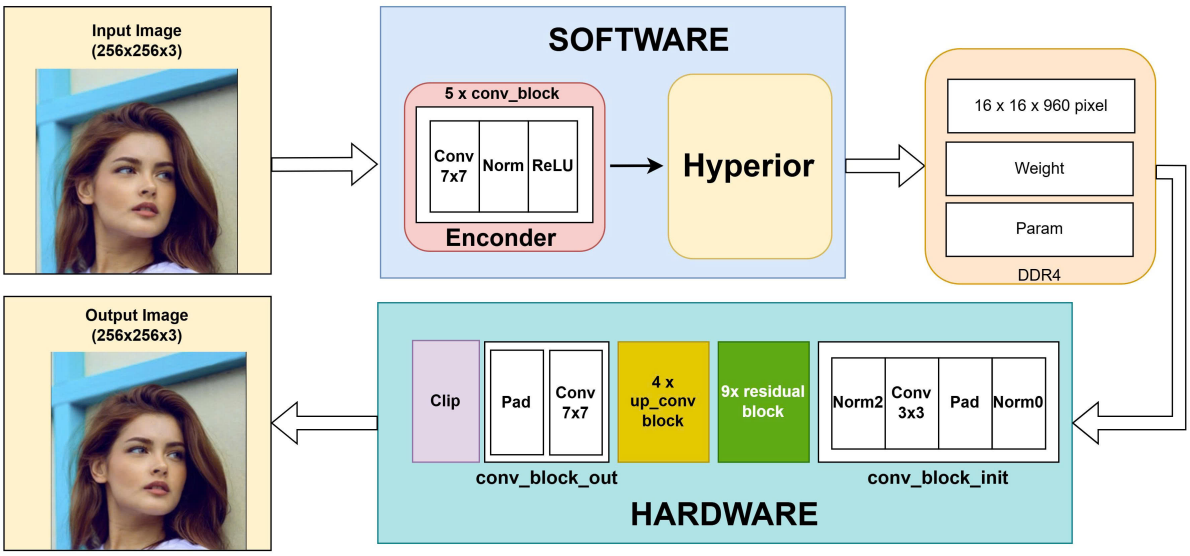

Figure 2.16 System-level hardware–software co-design overview of the HiFiC inference pipeline.

2.9.0.1 System-Level Role of the ResBlock Accelerator.

As shown in Figure 2.16, the ResBlock accelerator is embedded within the Generator stage of the HiFiC model, which dominates the overall inference workload. The Encoder and Hyperprior components are executed entirely in software on the PS, while the Generator is partitioned to offload its most computationally intensive portion—the residual blocks—to hardware.

The Generator pipeline begins with an initial convolution block (`conv_block_init`) that transforms the latent representation into a fixed-size feature map of $16 \times 16 \times 960$. This tensor serves as the input to a sequence of nine identical residual blocks. Because all residual blocks share the same structure and tensor dimensions, the same ResBlock accelerator can be reused multiple times, invoked iteratively by the PS.

2.9.0.2 *Hardware–Software Functional Partitioning.*

Figure 2.16 clearly separates software and hardware responsibilities:

- **Software (PS):** Model control flow, Encoder execution, Hyperprior processing, parameter management, and orchestration of hardware accelerator invocations.
- **Hardware (PL):** High-throughput execution of ResBlock computations, including convolution, normalization, activation, and residual addition.

This partitioning allows the PS to retain flexibility in model execution while delegating performance-critical operations to specialized hardware.

2.9.0.3 *PS–PL Interaction Overview.*

The interaction between PS and PL follows a coarse-grained invocation model. Intermediate feature maps, convolution weights, and normalization parameters are stored in external DDR memory. For each ResBlock execution, the PS configures the accelerator through a lightweight control interface and triggers kernel execution. The PL reads input feature maps and parameters directly from DDR, performs the ResBlock computation, and writes the resulting feature map back to memory.

From the perspective of the PS, each ResBlock accelerator invocation behaves as an atomic operation. This abstraction simplifies software control logic and avoids fine-grained synchronization between individual CNN operators.

2.9.0.4 *Data Flow and Memory Organization.*

As depicted in Figure 2.16, all large tensors are exchanged through shared DDR memory. This design choice eliminates explicit streaming interfaces between software and hardware and enables straightforward integration with existing software frameworks. The hardware accelerator internally exploits on-chip buffers and pipelining to minimize repeated external memory access, while externally exposing a simple memory-mapped interface.

2.9.0.5 *Connection to Chapter 3.*

This section establishes the architectural context for the detailed system integration presented in Chapter 3. While this chapter focuses on accelerator design and architectural decisions, the next chapter elaborates on the concrete PS–PL communication mechanisms, driver-level control flow, and runtime execution on the target Zynq UltraScale+ platform.

Overall, Figure 2.16 demonstrates how the proposed ResBlock accelerator fits naturally into a hardware–software co-design strategy, accelerating the dominant workload

of the HiFiC Generator while preserving software flexibility and system-level simplicity.

2.10 High Level Synthesis

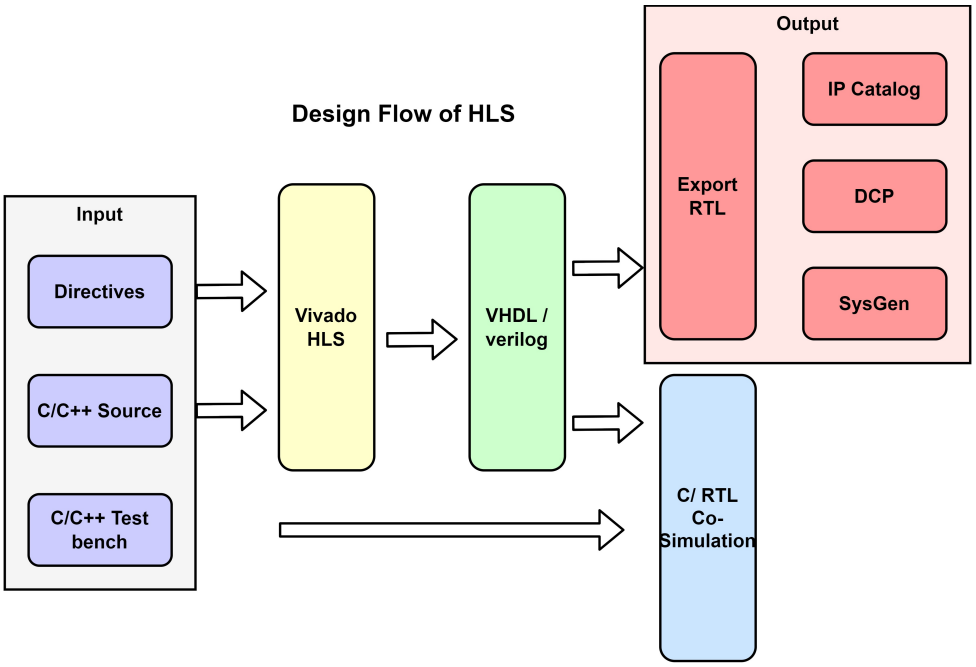

Figure 2.17 Design Flow of HLS

With HLS tools, developers can deploy algorithms and functions from a high-level language such as C or C++, reducing the complexity of the development process and optimizing the hardware performance. It serves various purposes, including:

Accelerating Development Process: Using a high-level programming language helps reduce development time compared to using low-level languages like Verilog or VHDL. Developers can focus more on algorithmic modeling rather than the details of hardware implementation.

Resource Optimization: HLS has automatic optimization capabilities, effectively utilizing hardware resources. This is particularly crucial when working on platforms with limited resources, such as FPGAs.

Flexible Control: Features like stream, AXIS, fixed-point, and pragma provide developers with flexible control during the automatic optimization process. This helps optimize hardware according to the specific requirements of the application.

1. **Stream**:

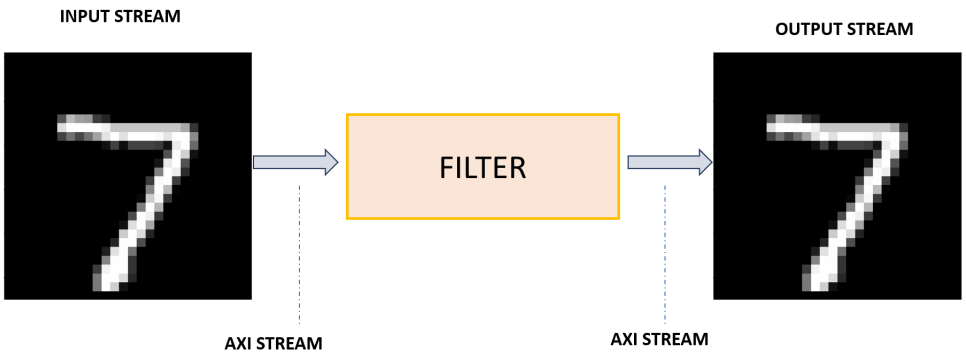

Figure 2.18 Stream HLS

Streaming is a data transfer method where data samples are sent sequentially from the first sample. Modeling designs using streaming data can be challenging in C. The use of pointers to perform multiple read and/or write accesses can introduce issues related to type qualifiers and test bench construction.

Vitis HLS provides a C++ template class ‘hls::stream<>‘ for modeling streaming data structures. These streams have the following attributes:

- In C code, ’hls::stream<>’ acts like an infinite-depth FIFO.
- They are read and written sequentially, meaning that once data is read from an ’hls::stream<>’, it cannot be read again.
- An ’hls::stream<>’ on the top-level interface is, by default, implemented with an ’ap_fifo’ interface for the Vivado IP flow or as an ‘axis‘ interface for the Vitis kernel flow.
- Streams can be named, and the depth of the FIFO can be adjusted.

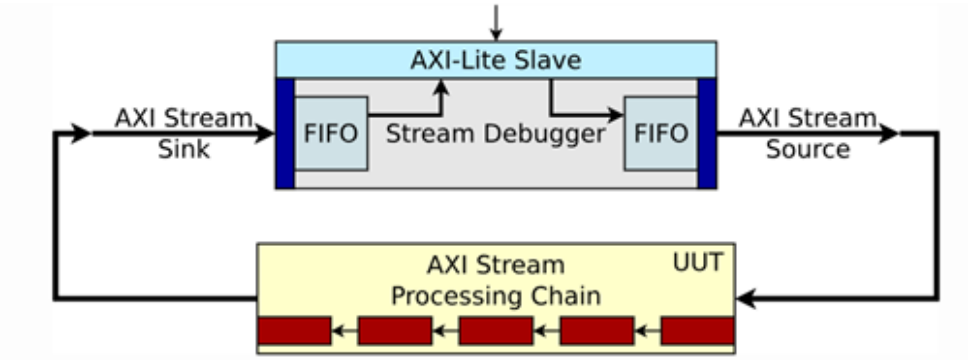

Figure 2.19 Stream HLS

In addition, my project utilizes the AXI Stream interface for handling these streams efficiently. The choice of the AXI Stream interface further enhances the communication between different hardware components, ensuring seamless and standard-

ized data transfer. This approach not only simplifies the implementation of read and write operations but also contributes to the overall optimization of computational processes within the project.

2. **Pragma**: Pragma plays a crucial role in guiding or modifying how High-Level Synthesis (HLS) tools organize and synthesize source code to optimize performance and utilize hardware resources. Below are some pragmas I use in my project:

   - Dataflow Pragma: #pragma HLS dataflow: This pragma is used to create a DATAFLOW region, where all implemented functions or source code blocks operate independently and in parallel. This helps optimize performance by allowing functions to perform computations without waiting.
   - Pipeline Pragma: #pragma HLS pipeline: This pragma indicates that loops in the source code can be divided into stages and executed in parallel. This helps optimize the working speed of the logic circuit.

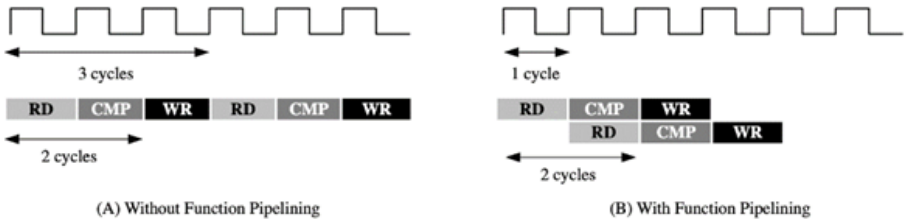

Figure 2.20 Pragma

   - Interface Pragma: #pragma HLS interface: This pragma is used to define the module's interface in the project, including AXI4-Stream, AXI4-Master, AXI4-Slave interfaces, and various other interface types.
   - Dependence Pragma: #pragma HLS dependence variable: This pragma is used to identify dependencies between variables, helping the HLS tool better understand the dependency relationships between data.

CHAPTER 3. FPGA IMPLEMENTATION

In this section, we implement the CNN algorithm in the previous section in FPGA known as the *Zynq™ UltraScale+™ MPSoC ZCU104* to evaluate with benmarks. This is a very complete board with a large number of capabilities intended to be used in a lot of different applications.

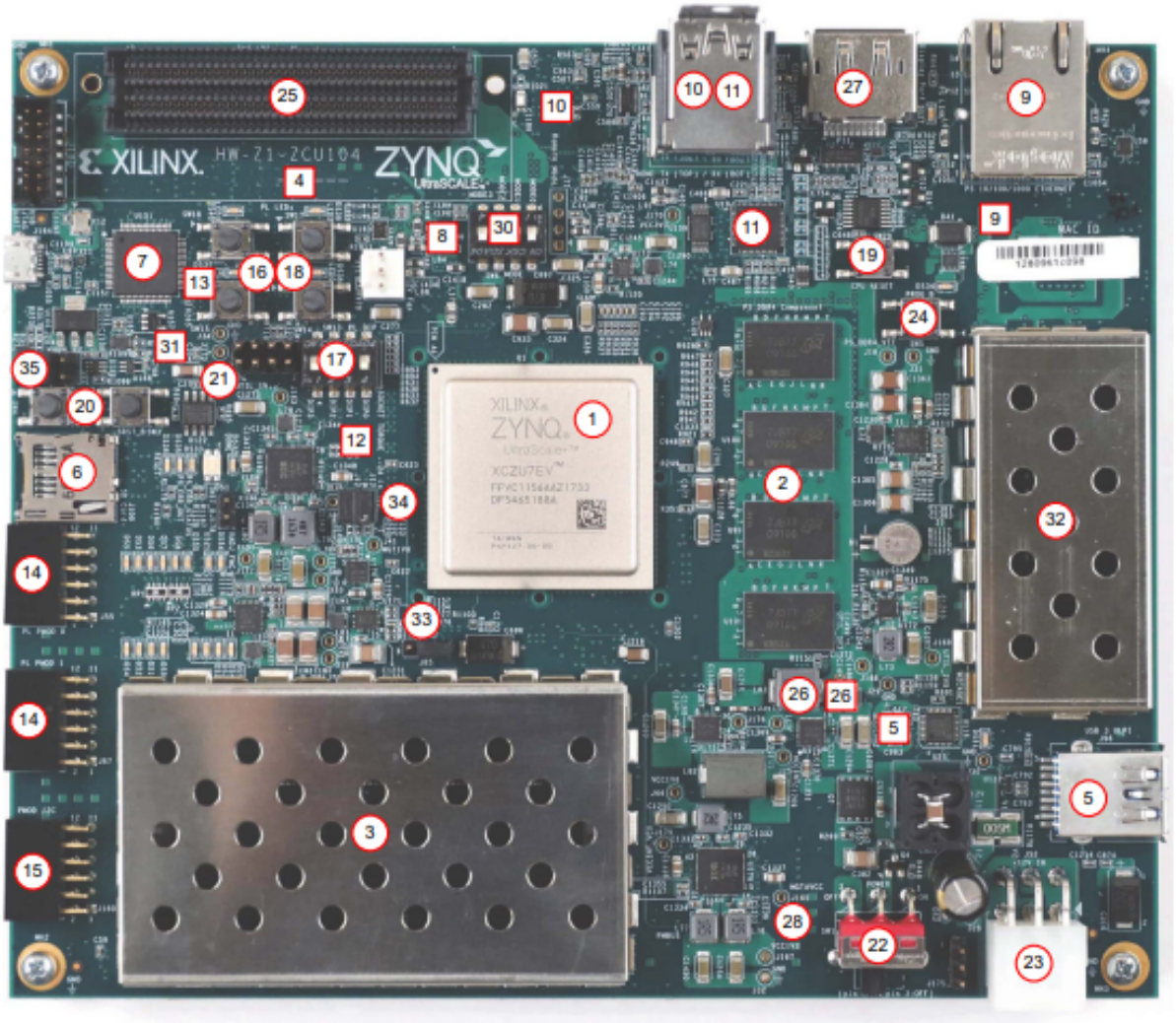

Figure 3.1 Zynq™ UltraScale+™ MPSoC ZCU104

With the most relevant for this project are listed in table.3.1:

1	Zynq UltraScale+ XCZU7EV MPSoC
3	PL-Side SODIMM DDR4
5	USB 3.0 Transceiver and USB 2.0 ULPI PHY
6	SD Card Interface connector
7	Programmable Logic JTAG
27	Display Port connector

Table 3.1 Relevant components for this project

3.1 System Design and Data Flow

Figure 3.2 illustrates the overall system architecture of the proposed design implemented on the ZCU104 evaluation board. The system follows a hardware–software co-design paradigm, where data acquisition, control, and I/O handling are managed by the Processing System (PS), while compute-intensive operations are offloaded to the Programmable Logic (PL).

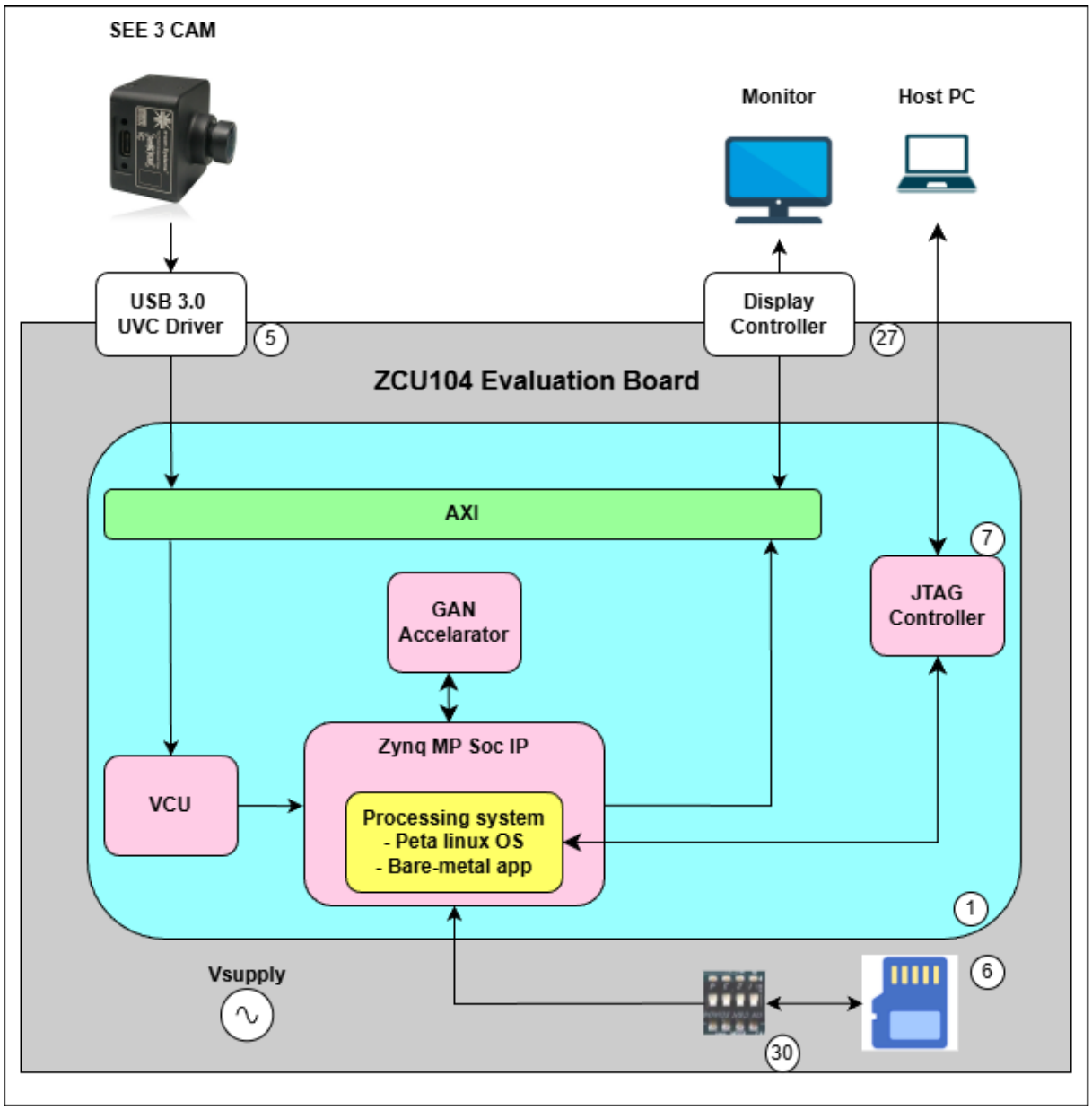

Figure 3.2 Overall system architecture on ZCU104

The input data is captured by a USB 3.0 camera and transmitted to the PS through the USB Video Class (UVC) driver. The video stream is handled by the embedded Linux environment running on the ARM Cortex-A53 cores, where GStreamer is used to manage the video pipeline. The incoming frames are first stored in the PS DDR memory via Direct Memory Access (DMA).

To accelerate the computationally intensive parts of the GAN-based image compression model, a dedicated GAN accelerator is implemented in the PL. The PS transfers input feature data to the PL through AXI interfaces, where the accelerator performs inference on selected components of the Generator network. After processing, the output data is written back to the PS DDR memory and forwarded to the Display Controller for

real-time visualization on an external monitor. The communication between the Display Controller and the monitor is handled using the *kmssink* plugin.

Figure 3.3 shows the detailed Vivado block design of the system. The Zynq UltraScale+ MPSoC IP serves as the central integration point, connecting the PS and PL domains through multiple AXI interconnects. The GAN accelerator is instantiated as a custom IP core in the PL and is accessed by the PS via AXI memory-mapped interfaces.

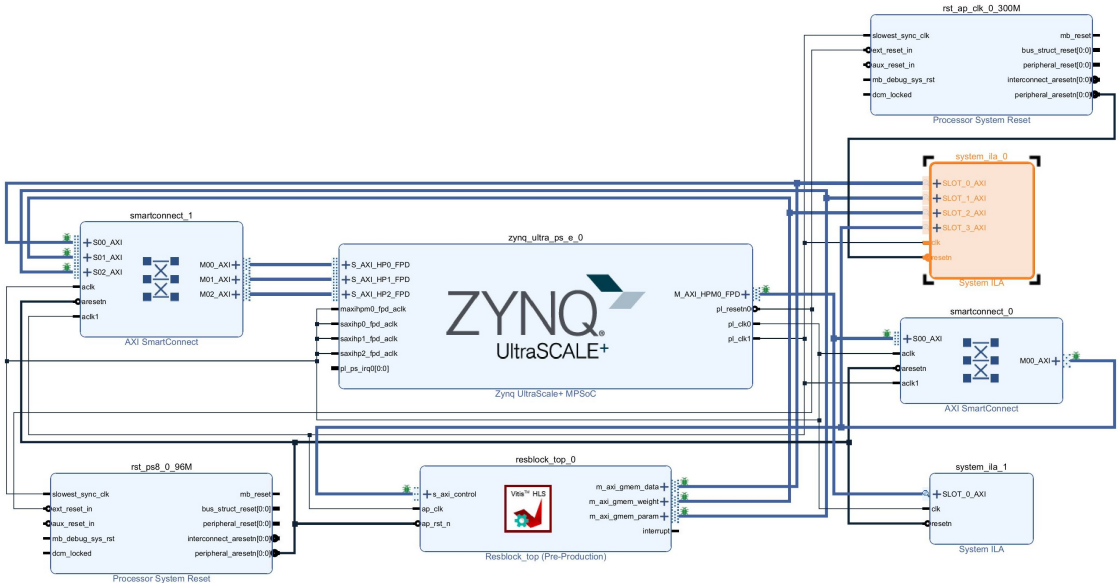

Figure 3.3 Vivado block design of the proposed system

The operating principle of the MPSoC differs from that of a standalone processor system. Instead of directly processing streaming data, the ARM Cortex-A53 cores retrieve data from peripheral interfaces through DMA transfers to the PS memory. The data is then dispatched to the PL for hardware acceleration. Once processing is completed, the results are transferred back to the PS memory and subsequently forwarded to peripheral devices such as the display interface.

In addition to the GAN accelerator, a Video Codec Unit (VCU) is integrated into the system to support video encoding and decoding. The VCU Encoder compresses the incoming video stream to reduce storage and memory bandwidth requirements. However, performing both encoding and decoding at high data rates (e.g., 60 frames per second at 1920×1080 resolution) can lead to memory bandwidth congestion when accessing the PS DDR.

To mitigate this issue, the following data management strategy is adopted:

- The data flow from the VCU Encoder to the PS DDR memory remains unchanged.
- The decoded data from the VCU Decoder is first written to the PL-side DDR mem-

ory through the VCU DDR4 Controller. The data is then transferred to the PS DDR memory using the PS DMA engine, reducing contention on the shared memory interface.

This system design enables efficient data movement, balances memory bandwidth usage between the PS and PL domains, and provides a scalable platform for accelerating GAN-based image compression on the ZCU104 MPSoC.

3.2 Integrating CNN into the Data Acquisition System

Since the input for CNN will be a single RGB frame rather than a video stream, there will be some changes compared to the data acquisition system described in section 3.1. In this section, the encoded frame data will be stored in the RAM of the APU block (consisting of 4 Arm Cortex A53 cores) instead of storing it in the DRAM of the PS.

Then, the data of this frame will be decoded, stored in the PS's DRAM, and the DMA block will be used to read the data from memory and transfer it to the CNN. Therefore, the block diagram will be as follows 3.4:

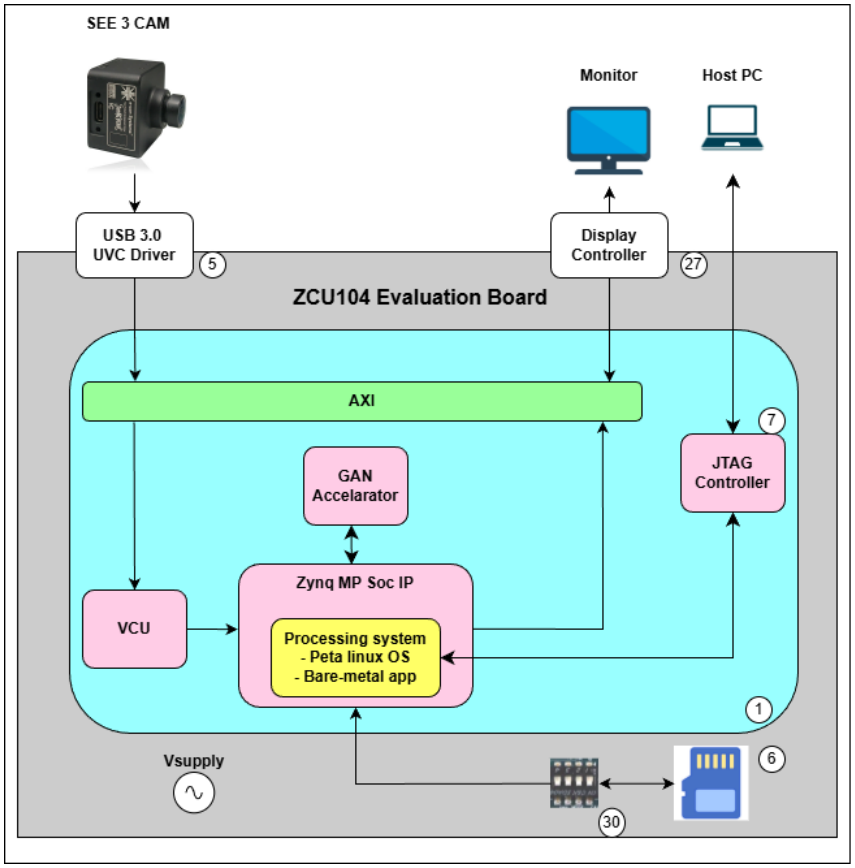

Figure 3.4 System block diagram

So the data flow when deploying Petalinux onto the system will consist of 4 stages:

1. Data captured from the camera is a video streamed into the DRAM on the PS. It is then sent to the VCU for encoding one frame and saved as an H264 encoded .ts file in the /home/ directory of Petalinux.

2. Utilizing the imread() function from OpenCV to read the .ts file saved in the /home/ directory and perform decoding. The data is then converted from YUV color space to BGR and then to RGB using the cvtColor() function. Finally, the decoded image is written into the /dev/mem directory using the imwrite() function.

3. Reading the decoded RGB image file from stage 2 and transferring it to the CNN through the DMA block.

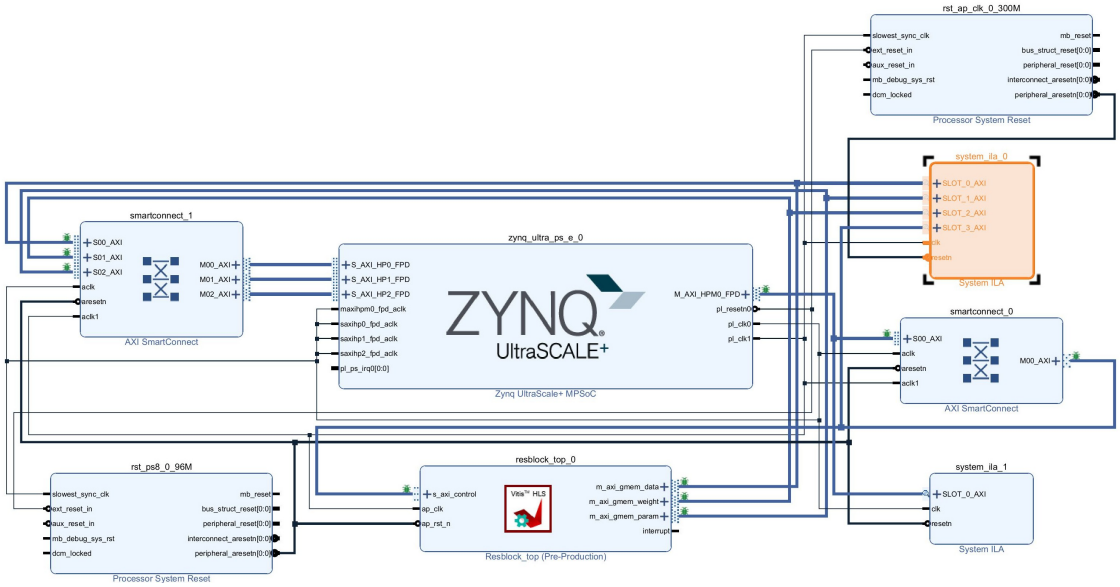

Figure 3.5 Diagram showing data flow

The .ts file is exported and saved in the /home/ directory for easy retrieval and comparison with the original image to calculate the compression ratio. After that, it is decoded and saved in /dev/mem/ to be sent to the CNN via DMA.

3.3 PetaLinux Software Environment

The ZCU104 evaluation board operates under an embedded Linux operating system based on PetaLinux. PetaLinux is selected due to its compatibility with the Zynq UltraScale+ MPSoC platform and its ability to tightly integrate hardware accelerators implemented in the Programmable Logic (PL) with software running on the Processing System (PS).

The embedded Linux environment runs on the ARM Cortex-A53 cores and is responsible for system initialization, peripheral management, and runtime control of

the hardware accelerator. A lightweight PetaLinux configuration is adopted to minimize software overhead while maintaining sufficient support for video processing and hardware communication.

Figure 3.6 illustrates the embedded Linux boot process on the ZCU104 platform. After power-on, the system follows a multi-stage boot sequence, starting from the on-chip BootROM and progressing through several software layers until the Linux user space becomes operational.

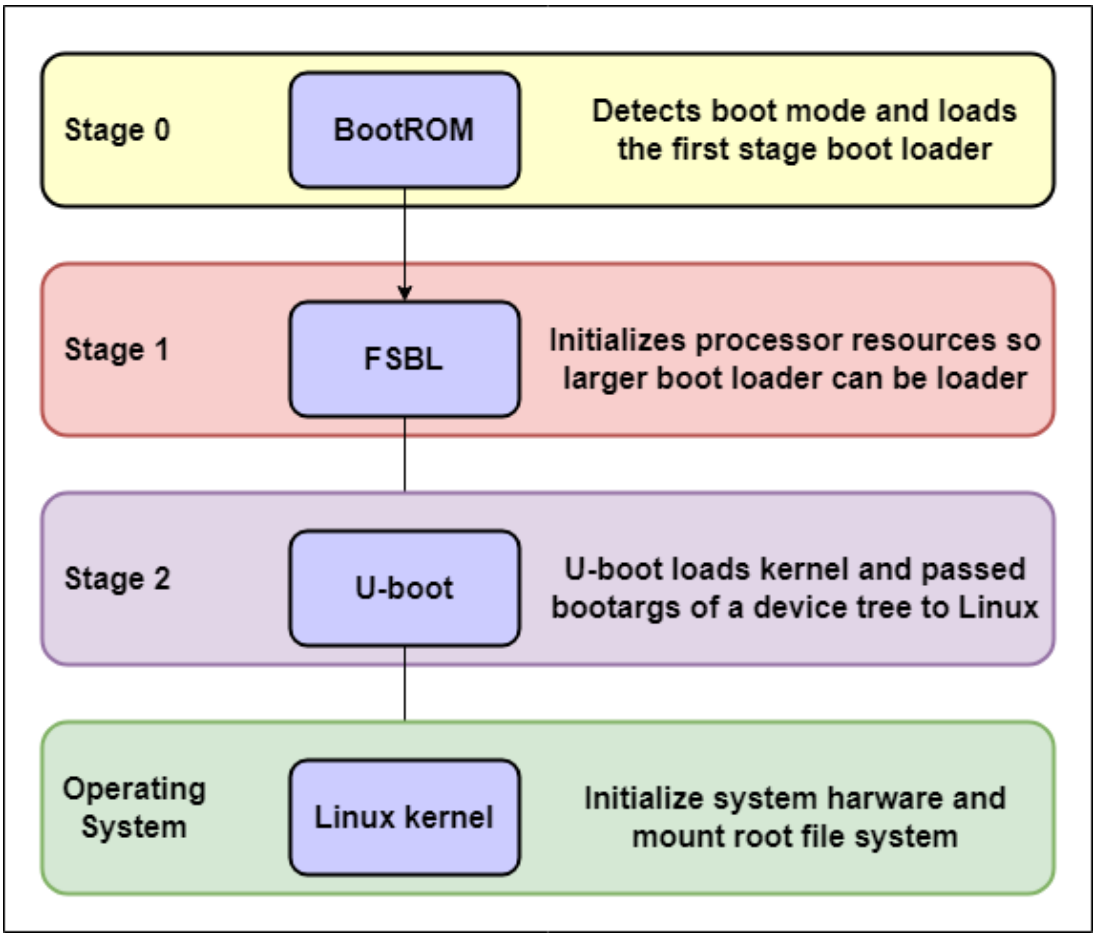

Figure 3.6 Embedded Linux boot process on Zynq UltraScale+ MPSoC

During the boot process, the First Stage Boot Loader (FSBL) initializes system clocks, external memory, and configures the Programmable Logic by loading the hardware bitstream. Subsequently, U-Boot loads the Linux kernel and the device tree, allowing the kernel to initialize device drivers and mount the root file system.

Once the PetaLinux system is fully booted, user-space applications are launched to manage video acquisition, memory allocation, and communication with the GAN accelerator in the PL. This software stack enables flexible control of data flow between peripherals, PS memory, and hardware accelerators while supporting rapid debugging and system-level performance evaluation.

3.4 Boot Process of the ZCU104 Platform

The boot process of the ZCU104 platform follows the standard boot sequence of the Zynq UltraScale+ MPSoC and is designed to initialize both the Processing System (PS) and the Programmable Logic (PL). The system boots from an SD card containing all required boot and system images.

The boot sequence consists of multiple stages:

- **Stage 0 – BootROM:** After power-on, the on-chip BootROM detects the boot mode configuration and loads the First Stage Boot Loader (FSBL) from the SD card.
- **Stage 1 – FSBL:** The FSBL initializes essential system resources, including clock configuration, memory controllers, and the Programmable Logic. The PL bitstream generated by Vivado is loaded at this stage to configure the hardware accelerators.
- **Stage 2 – Platform Management and Secure Firmware:** Platform Management Unit (PMU) firmware and ARM Trusted Firmware (ATF) are loaded to manage power, reset, and security-related functions of the MPSoC.
- **Stage 3 – U-Boot:** U-Boot is responsible for loading the Linux kernel and the device tree blob into memory. It also prepares the system environment for kernel execution.
- **Stage 4 – Linux Kernel:** The Linux kernel initializes hardware drivers, mounts the root file system, and starts user-space processes.

All required boot components are packaged into three main files stored on the SD card:

- **BOOT.BIN:** Contains the FSBL, PL bitstream, PMU firmware, ATF, and U-Boot.
- **Image.ub:** Includes the Linux kernel and device tree.
- **Root File System:** Provides user-space libraries, drivers, and applications.

After the boot process is completed, the PetaLinux operating system becomes fully operational. User-space applications are then launched to control video acquisition, manage data transfers, and coordinate the execution of the GAN accelerator in the Programmable Logic.

CHAPTER 4. RESULTS

4.1 Quantization Results and Evaluation

This section presents the experimental evaluation of the mixed-precision quantization applied to the HiFiC-Hi model. The objective is to assess the impact of transitioning from full-precision FP32 computation to a mixed-precision FP16 implementation on compression efficiency and reconstructed image quality.

Experiments were conducted on a representative test image with a resolution of 256×256. Two configurations were compared under identical conditions: the baseline FP32 model and the quantized mixed-precision FP16 model, where internal computations operate in FP16 while input and output interfaces remain in FP32.

Table 4.1 summarizes the quantitative compression metrics obtained from both models.

Table 4.1 Comparison between FP32 and FP16 mixed-precision HiFiC models

Metric	FP32	FP16 Mixed	Delta
Original Size (bytes)	25,879	25,879	0
Bitstream Size (bytes)	6,522	6,517	-5
Output Image Size (bytes)	97,596	97,496	-100
Compression Ratio	3.97×	3.97×	0.00
Space Saving (%)	74.80	74.82	+0.02
Bits Per Pixel (bpp)	0.7961	0.7955	-0.0006
PSNR (dB)	19.08	19.05	-0.03
SSIM	0.8315	0.8314	-0.0001
MSE	804.315755	808.560496	4.244

The results indicate that the compression performance of the HiFiC model is preserved after quantization. The bitstream size is slightly reduced, while the compression ratio and space-saving metrics remain unchanged. This suggests that the probability distribution of the latent representation is not significantly affected by FP16 rounding errors, allowing the entropy coding process to operate efficiently.

From a perceptual quality perspective, the degradation introduced by mixed-precision computation is negligible. The observed PSNR reduction of 0.03 dB is well below the threshold of perceptual visibility, and the SSIM metric remains nearly identical between the two configurations. These results confirm that structural features and texture details are effectively preserved under FP16 computation.

Figure 4.1 provides a visual comparison between the reconstructed images produced by the FP32 and FP16 models.

Figure 4.1 Visual comparison of reconstructed images: FP32 model (left) and FP16 mixed-precision model (right)

No visible artifacts or structural distortions are observed in the quantized output. Overall, the experimental results demonstrate that the proposed mixed-precision quantization strategy achieves a favorable balance between numerical efficiency and output fidelity, validating its suitability for FPGA deployment of the HiFiC image compression model.

4.2 Verification

After completing the coding and simulation process, we proceeded to synthesize the source code to evaluate the performance and resource utilization of the circuit.

4.2.1 Generative Adversarial Network

4.2.1.1 Results after Synthesis

We obtained results after synthesizing the GAN circuit, including: DSP, FF, LUT, and DRAM Utilization Parameters in the GAN Block:

Table 4.2 Utilization Report

Name	CLB LUTs (230400)	CLB Regs (460800)	CARRY8 (28800)	F7 Mux (115200)	F8 Mux (57600)	CLB (28800)	LUT Logic (230400)	LUT Mem (101760)	BRAM Tile (312)	URAM (96)	DSPs (1728)	BUFGCE (208)
design_1_wrapper	57291	73774	1579	1695	724	12198	52406	4885	105	76	48	2
dbg_hub	447	753	7	0	0	135	427	20	0	0	0	1
design_1_i	56844	73021	1572	1695	724	12066	51979	4865	105	76	48	1

We continued evaluating and documenting the resource usage in the GAN block, ensuring optimal resource utilization. We measured and verified the processing

time parameters of the GAN circuit.

4.2.1.2 Main Blocks of the GAN Circuit

We listed and described the main blocks in the GAN circuit, providing a clear understanding of the structure and interactions between components.

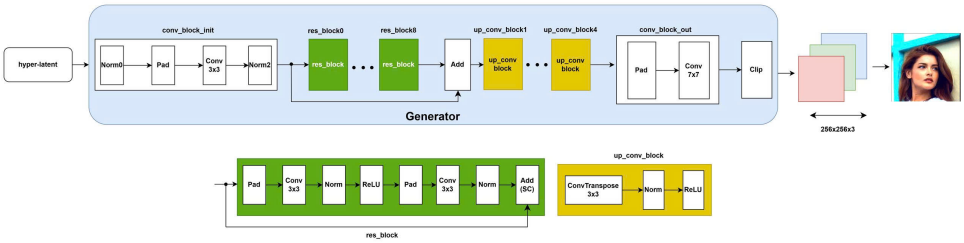

Figure 4.2 Main GAN block

This information offers a detailed and comprehensive view of the performance, resource utilization, and structure of GAN after the synthesis and simulation process. This data serves as a foundation for further optimization and adjustments to the circuit if necessary.

4.2.1.3 GAN Results

We created waveforms to visualize the correspondence between the input and output data during the generating process. The following aspects were observed and analyzed. We compared the input data with the corresponding output data, ensuring that the decoding process produced the expected results.

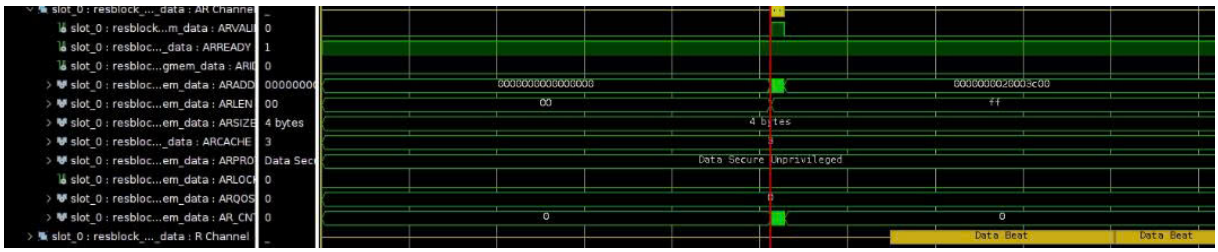

Figure 4.3 Input Data

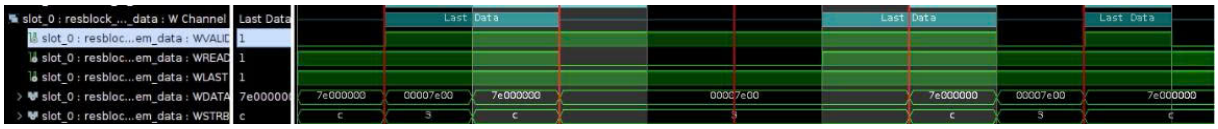

Figure 4.4 Output Data

4.3 Compare with python

Next, we will compare the simulation results using orginal and quantized models based on the key metric PSNR.

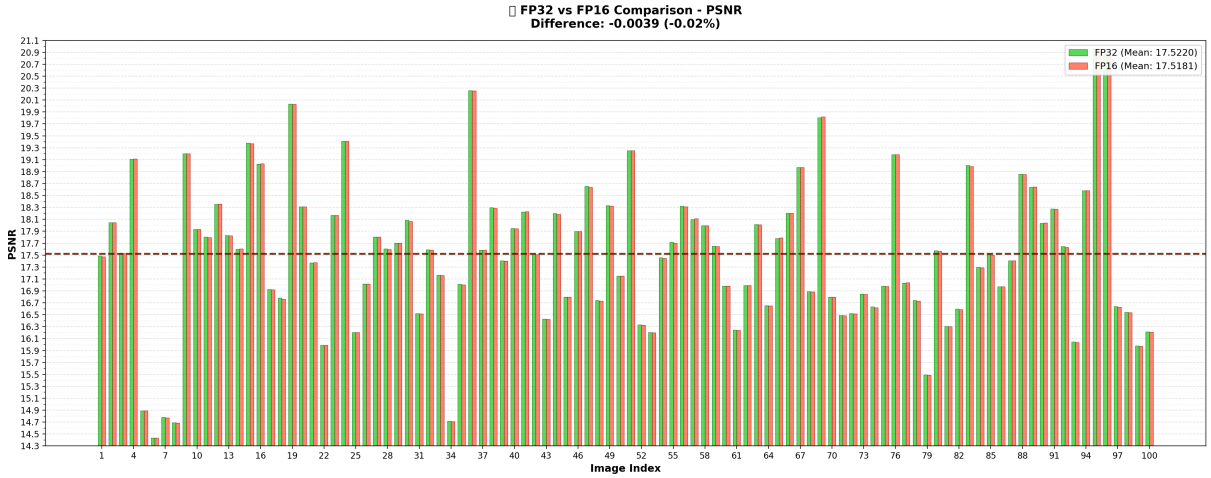

Figure 4.5 Comparison between original and our quantized model

Figure 4.5 depicts the PSNR results of 100 images using Python original model and 100 images using quantized model HLS. The results also demonstrate the similarity between the Python model and the HLS simulation. Importantly, our model utilizes fixed-point representation and has been tested on ZCU104.

4.4 Implementaion on ZCU104

After creating SoC Block of the all system, we synthesis generate bitstream and implement on ZCU104 through Vivado. We, then create application on Vitis to calculate the end-to-end latency of our model and get the result approximately five times faster than software measurement with the frequency of 300Mhz.

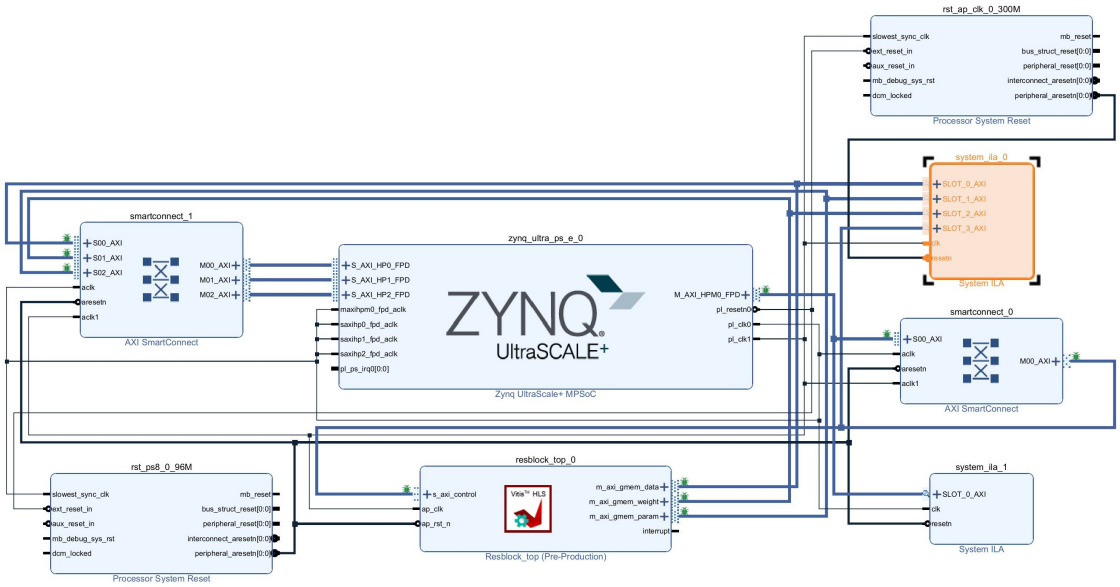

Figure 4.6 Block design for FPGA

CHAPTER 5. CONCLUSIONS AND FUTURE DEVELOPMENT

The Generative Adversarial Network (GAN) project for image compression and reconstruction has yielded highly positive results, demonstrating the superior capability of adversarial learning in optimizing both visual representation and synthesis. A standout achievement of this research is the successful transition from algorithmic theory to high-performance implementation. Experimental results indicate that the hardware-accelerated version of the model achieves a processing speed five times faster than the software-based execution. This significant leap in performance confirms that while the GAN model excels in preserving high-frequency textures and perceptual realism, its integration into specialized hardware provides the necessary throughput for real-time applications. The project effectively bridges the gap between deep learning complexity and practical efficiency, showcasing a robust and adaptable solution for next-generation image processing.

Moving forward, the project will focus on several key trajectories to transition the GAN model from a research prototype to a robust, real-world solution. A primary objective is architectural refinement, aimed at stabilizing the adversarial training process and mitigating issues such as mode collapse to ensure consistent output quality. To bridge the gap between software and hardware, we plan to optimize the model for real-time deployment on embedded systems or FPGA platforms, specifically targeting reduced computational complexity and latency. Furthermore, the scope of the project will expand to handle ultra-high-resolution datasets and diverse visual domains, ensuring scalability and robustness. We also intend to explore hybrid modeling by integrating GANs with Vision Transformers or Diffusion Models to push the boundaries of reconstruction fidelity. Ultimately, these advancements will integrate enhanced security measures and optimized bit-rate allocation, transforming the model into a versatile tool for high-efficiency, secure image transmission in bandwidth-constrained environments.

REFERENCES

[1] F. Mentzer, G. Toderici, M. Tschannen, and E. Agustsson, "High-fidelity generative image compression," *arXiv preprint arXiv:2006.09965*, 2020.